

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
BACHAREL EM CIÊNCIAS DA COMPUTAÇÃO

Tális Breda

ESTUDO DA FAMÍLIA DE EMPARELHAMENTO EM GRAFOS

Florianópolis
2024

Tális Breda

ESTUDO DA FAMÍLIA DE EMPARELHAMENTO EM GRAFOS

Trabalho de Conclusão do Curso de Graduação em Ciências da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina para a obtenção do título de Bacharel em Ciências da Computação.
Orientador: Prof. Dr. Rafael de Santiago

Florianópolis
2024

RESUMO

Problemas de emparelhamento em grafos, definidos como uma seleção de conjuntos de arestas sem vértices em comum, possuem ampla relevância em áreas da computação, como visão computacional, e também fora dela, como na biologia. São bastante importantes em tarefas que envolvem correspondência de elementos, como a comparação e correspondência de elementos em conjuntos distintos, por exemplo em análise de dados estruturados. Enquanto as formulações clássicas são solúveis eficientemente, variações com restrições adicionais pertencem frequentemente à classe NP-Difícil, exigindo métodos inovadores para equilibrar precisão e eficiência. Considerando a quantidade de algoritmos e métodos presentes na literatura, torna-se difícil encontrar a maneira mais eficiente de resolver um problema específico. Este trabalho busca levantar, comparar e classificar métodos computacionais, destacando suas aplicações e limitações, com o objetivo de fornecer insights úteis para pesquisadores e profissionais na escolha de soluções eficazes para cenários práticos.

Palavras-chave: Emparelhamento em grafos, grafos bipartidos, visão computacional

ABSTRACT

Graph matching problems, defined as a selection of edges sharing no common vertices, hold significant relevance in computational fields such as computer vision, as well as in other domains like biology. They are crucial in tasks involving element correspondence, such as the comparison and matching of elements across distinct sets—for instance, in the analysis of structured data. While classical formulations are efficiently solvable, variations with additional constraints often belong to the NP-Hard class, necessitating innovative methods to balance accuracy and efficiency. Given the vast number of algorithms and methods available in the literature, identifying the most efficient approach for a specific problem becomes challenging. This work aims to survey, compare, and classify computational methods, highlighting their applications and limitations, with the goal of providing valuable insights to assist researchers and practitioners in selecting effective solutions for practical scenarios.

Keywords: Graph matching, bipartite graphs, computer vision

Conteúdo

1 Introdução **8**
1.1 Objetivos 8
1.1.1 Objetivo geral 8
1.1.2 Objetivos específicos 8
1.2 Delimitação do estudo 9

2 Fundamentação teórica **9**
2.1 Definições sobre grafos 9
2.1.1 Grafo direcionado 9
2.1.2 Grafo ponderado 9
2.1.3 Grafo bipartido 9
2.2 Busca em grafos 10
2.2.1 Representação computacional 10
2.2.2 Busca em largura 10
2.2.3 Busca em profundidade 10
2.3 Caminhos mais curtos 11
2.3.1 Princípio do relaxamento e subestrutura ótima 11
2.3.2 Algoritmo de Dijkstra 11
2.3.3 Algoritmo de Bellman-Ford 12
2.3.4 Algoritmo de Floyd-Warshall 12
2.4 Redes de fluxo em grafos 13
2.4.1 Definições básicas 13
2.4.2 O problema do fluxo máximo 13
2.4.2.1 Corte 13
2.4.2.2 Redes residuais 14
2.4.2.3 Caminhos aumentantes 14
2.4.3 Fluxo de custo mínimo 14
2.4.3.1 Condições de otimalidade e ciclos negativos 15
2.4.3.2 Potenciais de vértices e custos reduzidos 15
2.5 Problema Geral do Emparelhamento 15
2.5.1 Matching em grafos bipartidos 16
2.5.2 Formulação geral (emparelhamento e otimização) 16

3 Problemas de emparelhamento em grafos bipartidos **16**
3.1 Emparelhamento de cardinalidade máxima 16
3.1.1 Descrição do problema 16
3.1.2 Propriedades 16
3.1.2.1 Emparelhamento máximo vs maximal 16
3.1.2.2 Caminhos aumentantes 16
3.1.2.3 Teorema de Kőnig 17
3.1.2.4 Teorema de Hall 17
3.1.2.5 Teorema de Gallai e Conjunto Independente Máximo 17
3.1.3 Algoritmos exatos 18
3.1.3.1 Algoritmo do caminho aumentante 18
3.1.3.2 Redução ao problema de fluxo máximo 19
3.1.3.3 Algoritmo de Edmonds-Karp 19
3.1.3.4 Algoritmo de Dinic 21
3.1.3.5 Algoritmo de Hopcroft-Karp 21
3.1.4 Heurísticas 23
3.1.4.1 Heurística gulosa simples 23
3.1.4.2 Heurística de Karp-Sipser 24
3.2 Problema de atribuição (Assignment Problem) 26

3.2.1 Descrição do problema . . . 26
3.2.2 Propriedades . . . 26
3.2.3 Algoritmos exatos . . . 26
3.2.3.1 Método Húngaro . . . 26
3.2.3.2 Jonker-Volgenant . . . 28
3.2.3.3 Redução para problema de fluxo de custo mínimo . . . 29
3.2.4 Heurísticas . . . 31
3.2.4.1 Heurística gulosa ordenada . . . 31
3.3 Problema de atribuição gargalo . . . 32
3.3.1 Descrição do problema . . . 32
3.3.2 Soluções exatas . . . 32
3.3.2.1 Métodos de threshold . . . 32
3.3.2.2 Método dual . . . 33
3.3.2.3 Método de caminhos aumentantes . . . 34
3.3.3 Heurísticas . . . 36
3.3.3.1 Heurística Construtiva Gulosa . . . 36
3.4 Problema do emparelhamento estável . . . 37
3.4.1 Descrição do problema . . . 37
3.4.2 Propriedades . . . 37
3.4.3 Algoritmo de Gale-Shapley . . . 38

4 Problemas de emparelhamento em grafos gerais . . . 38
4.1 Emparelhamento de cardinalidade máxima . . . 39
4.1.1 Descrição . . . 39
4.1.2 Propriedades . . . 39
4.1.2.1 Ciclos ímpares . . . 39
4.1.2.2 Flor (Blossom) . . . 39
4.1.3 O algoritmo de Edmonds . . . 39
4.1.3.1 Teorema de Tutte-Berge . . . 43
4.1.4 Heurísticas . . . 43
4.1.4.1 Algoritmo de crescimento de caminho (Path Growing) . . . 43
4.2 Emparelhamento ponderado máximo . . . 44
4.2.1 Formulação poliédrica e Restrições de Blossom . . . 44
4.2.2 A abordagem Primal-Dual . . . 45
4.2.3 O algoritmo de Edmonds Ponderado . . . 45
4.2.4 Otimizações . . . 46
4.2.5 Heurísticas . . . 46
4.2.5.1 Algoritmo de Crescimento de Caminho (Path Growing) . . . 46
4.3 Emparelhamento induzido máximo . . . 47
4.3.1 Propriedades . . . 48
4.3.2 Complexidade computacional . . . 48
4.3.3 Redução ao Problema do Conjunto Independente . . . 48
4.3.4 Abordagens de solução e heurísticas . . . 48
4.3.4.1 Algoritmo exato . . . 49
4.3.4.2 Heurísticas gulosas . . . 49
4.3.4.3 Metaheurísticas . . . 50
4.4 Outros problemas . . . 50

5 Metodologia . . . 51
5.1 Arquitetura do ambiente de testes . . . 51
5.2 Interface padrão dos problemas . . . 51
5.3 Geradores de instância . . . 51
5.4 Protocolo de medição . . . 52
5.5 Critérios de comparação . . . 52

5.6 Ambiente de execução e reprodutibilidade 53
5.7 Origem, adaptação e verificação dos algoritmos 53

6 Resultados e discussão 55
6.1 Cardinalidade máxima em grafos bipartidos 55
6.2 Atribuição linear 57
6.3 Atribuição gargalo 59
6.4 Casamento estável 60
6.5 Emparelhamento em grafos gerais 61
6.6 Emparelhamento induzido máximo 63
6.7 Defeitos em implementações públicas 64
6.8 Crescimento empírico frente à complexidade teórica 64

7 Trabalhos futuros 67

1 Introdução

Na teoria dos grafos e otimização combinatória, um emparelhamento em um grafo não-dirigido $G = (V, E)$ é definido formalmente como um subconjunto de arestas $M \subseteq E$ tal que nenhuma aresta em M compartilhe um vértice comum. Em termos precisos, para todo vértice $v \in V$, o grau de v no subgrafo induzido por M é no máximo 1. Esta definição fundamental é apresentada por Lovász e Plummer (1986) em sua obra seminal sobre a teoria dos emparelhamentos, onde também se estabelece a distinção clássica entre emparelhamentos em grafos bipartidos — cujos vértices podem ser divididos em dois conjuntos disjuntos U e V — e em grafos gerais.

Os problemas de emparelhamento têm aplicações significativas em várias áreas, como biologia computacional, onde podem ser empregados na diferenciação e classificação de estruturas proteicas (TAYLOR, 2002); redes sociais, para identificação de comunidades ou grupos (FAN, 2012); e visão computacional, para correspondências entre elementos de conjuntos distintos, como pontos em imagens ou vértices em malhas tridimensionais (HALLER et al., 2022).

É crucial, no entanto, fazer uma distinção precisa quanto à complexidade computacional desses problemas. Ao contrário do que uma análise superficial poderia sugerir, o problema de encontrar um emparelhamento de cardinalidade ou peso máximo pertence à classe de complexidade **P** (tempo polinomial), sendo solúvel por algoritmos clássicos como o Método Húngaro ou o Algoritmo de *Blossom* de Edmonds (SCHRIJVER, 2004). O verdadeiro desafio computacional, frequentemente classificado como **NP-Difícil**, surge em variações estruturais mais complexas, como o Emparelhamento Tridimensional ou o Problema da Atribuição Quadrática (QAP). Segundo Garey e Johnson (1990) e Burkard, Dell'Amico e Martello (2009), o QAP modela situações onde o custo depende da interação entre os pares escolhidos, tornando métodos exatos inviáveis para grandes instâncias e exigindo o uso de relaxações ou heurísticas avançadas.

Diante desse cenário, este trabalho visa sistematizar os principais problemas e métodos computacionais associados ao emparelhamento em grafos. Ele busca oferecer uma comparação detalhada entre suas aplicações e limitações, contribuindo para o entendimento aprofundado das ferramentas disponíveis e identificando oportunidades para o desenvolvimento de novas soluções. Essa abordagem torna-se ainda mais relevante à luz da importância prática desses algoritmos para resolver problemas reais em grande escala, conforme evidenciado por estudos recentes (SUSSMAN; LI, 2021).

1.1 Objetivos

1.1.1 Objetivo geral

Este trabalho tem como objetivo principal levantar e organizar os principais problemas de emparelhamento em grafos e os métodos computacionais presentes na literatura para resolvê-los, implementando esses métodos e comparando-os experimentalmente quanto ao desempenho e à qualidade das soluções.

1.1.2 Objetivos específicos

- Mapear as variações e características dos principais problemas de emparelhamento em grafos presentes na literatura.
- Classificar esses problemas e os métodos computacionais que os resolvem, segundo critérios como a estrutura do grafo, o objetivo de otimização e o tipo de método (exato, aproximado ou heurístico).
- Implementar os algoritmos selecionados, adaptando implementações públicas ou, quando não houver implementação adequada, partindo da descrição bibliográfica.
- Criar um ambiente de testes unificado e, nele, comparar os algoritmos quanto ao tempo de execução, à complexidade assintótica de tempo e espaço e à qualidade das soluções encontradas.
- Organizar os resultados em um formato que facilite o acesso e a compreensão dos tópicos abordados por pesquisadores e profissionais.

1.2 Delimitação do estudo

Os problemas de emparelhamento em grafos podem ser classificados de diversas maneiras, como sendo em grafos bipartidos ou não-bipartidos, ponderados ou não-ponderados, direcionados ou não-direcionados, problemas em multigrafos, problemas em hipergrafos, entre outros. Este trabalho focará em problemas que envolvem grafos menos complexos, excluindo multigrafos e hipergrafos.

2 Fundamentação teórica

2.1 Definições sobre grafos

Um **grafo** G é uma estrutura definida por um par $G = (V, E)$, consistindo de um conjunto finito V de elementos chamados **vértices** (ou **nós** ou **pontos**) e um conjunto (ou família) E de pares não ordenados de vértices, chamados de **arestas** (JUNGNICKEL, 2008). O conjunto V é o conjunto de vértices de G e E é a família de arestas de G.

No contexto deste projeto, assumimos que as arestas $e = (u, v)$ são pares não ordenados de vértices distintos. Se uma aresta e conecta os vértices a e b, diz-se que a e b são **adjacentes** ou **vizinhos**, e que a aresta e é **incidente** a a e b (SCHRIJVER, 2004).

O conceito de grafo é amplamente utilizado como uma representação abstrata concisa para estruturas complexas e para codificar relações binárias entre um conjunto de objetos.

2.1.1 Grafo direcionado

Se as relações entre os vértices forem assimétricas, utilizamos um **grafo direcionado** ou **dígrafo**, $D = (V, A)$. Neste caso, A é uma família de pares ordenados de vértices, chamados **arcos** (ou arestas direcionadas). Para um arco $e = (u, v)$, u é chamado o vértice inicial ou **cauda** (tail), e v é o vértice final ou **cabeça** (head) (MANBER, 1989)

2.1.2 Grafo ponderado

Um grafo $G = (V, E)$ é chamado **grafo ponderado** (ou com pesos) se uma **função de peso** (ou função de custo, ou função de comprimento) $w : E \to R$ está associada às arestas. Formalmente: $G = (V, E, w)$, onde $w : E \to R$. Geralmente, o peso $w(e)$ de uma aresta e é um valor real não negativo que representa o custo, comprimento ou capacidade associada àquela aresta (ROSEN; KREHER; STINSON, 1998).

Grafos ponderados podem ser representados por matrizes de adjacência de valor real A, onde a entrada A_{ij} é o peso w_{ij} da aresta (ZASLAVSKIY; BACH; VERT, 2009).

2.1.3 Grafo bipartido

Um grafo bipartido é um grafo não direcionado que pode ser facilmente colorido com apenas duas cores (MANBER, 1989).

Formalmente, um grafo $G = (V, E)$ é chamado **grafo bipartido** se o conjunto de vértices V puder ser particionado em dois subconjuntos disjuntos V_1 e V_2, chamados classes de cor, tais que:

$$V = V_1 \cup V_2 \quad e \quad V_1 \cap V_2 = \emptyset \tag{1}$$

e todas as arestas $e \in E$ conectam um vértice em V_1 a um vértice em V_2. Ou seja, não existem arestas que conectem vértices dentro do mesmo subconjunto (DASGUPTA; VAZIRANI, 2006).

Observações:

- Um grafo é bipartido se e somente se não contém ciclos de comprimento ímpar (SCHRIJVER, 2004)
- O grafo bipartido completo $K_{m,n}$ possui m vértices em V_1 e n vértices em V_2, e contém todas as arestas possíveis entre V_1 e V_2 (JUNGNICKEL, 2008)

2.2 Busca em grafos

A exploração sistemática dos vértices e arestas de um grafo é uma sub-rotina fundamental para a maioria dos algoritmos em redes, incluindo aqueles utilizados para encontrar emparelhamentos, fluxo máximo e componentes conexos. A busca em grafos consiste em seguir as arestas do grafo para visitar os vértices, permitindo descobrir a estrutura da rede e propriedades de conectividade (CORMEN et al., 2009).

2.2.1 Representação computacional

Antes de discutir os algoritmos de busca, é essencial definir como o grafo $G = (V, E)$ é representado computacionalmente, pois isso impacta diretamente a complexidade temporal das operações. Destacam-se duas representações primárias (AHO; HOPCROFT; ULLMAN, 1974):

- **Matriz de adjacência:** Uma matriz A de dimensão $|V| \times |V|$, onde $A_{ij} = 1$ se existe uma aresta $(i, j) \in E$, e 0 caso contrário. Embora permita verificação de existência de arestas em tempo constante $O(1)$, ela consome espaço $\theta(V^2)$,sendo ineficiente para grafos esparsos.
- **Lista de adjacência:** Consiste em uma lista de $|V|$ listas, onde cada lista $Adj[u]$ contém todos os vértices v tal que $(u, v) \in E$. Esta representação é preferível para grafos esparsos, pois consome espaço $\theta(V + E)$ (CORMEN et al., 2009).

Para os algoritmos de busca descritos a seguir, assume-se geralmente o uso de listas de adjacência, resultando em complexidades lineares em relação ao tamanho do grafo.

2.2.2 Busca em largura

A **busca em largura**, ou Breadth-First Search (BFS) é um dos algoritmos mais simples e fundamentais para a exploração de grafos. A BFS explora o grafo em ”camadas” ou níveis. Dado um vértice fonte s, o algoritmo visita primeiro todos os vizinhos de s (camada L_1), depois os vizinhos desses vizinhos (camada L_2), e assim sucessivamente (KLEINBERG; TARDOS, 2005).

O procedimento utiliza uma estrutura de dados do tipo Fila (FIFO) para gerenciar a fronteira de exploração. Cormen et al. (2009) descrevem o uso de um sistema de ”cores” para monitorar o estado dos vértices: brancos (não descobertos), cinzas (descobertos, mas com vizinhos ainda não explorados) e pretos (totalmente explorados).

Propriedades e Complexidade: A propriedade mais importante da BFS, destacada por Dasgupta e Vazirani (2006), é que ela calcula o caminho mais curto (em número de arestas) de s a todos os outros vértices alcançáveis em grafos não ponderados. A complexidade de tempo da BFS é $O(V + E)$, pois cada vértice é enfileirado no máximo uma vez e cada lista de adjacência é varrida uma única vez.

2.2.3 Busca em profundidade

A **busca em profundidade** ou Depth-First Search (DFS), ao contrário da estratégia por níveis da BFS, a busca em profundidade explora as arestas a partir do vértice mais recentemente descoberto que ainda possui arestas inexploradas. Conforme Manber (1989), a DFS ”aprofunda-se”no grafo tanto quanto possível e, quando não há mais para onde ir, realiza o *backtracking* (retrocesso) para explorar outros caminhos.

Computacionalmente, a DFS pode ser implementada de forma recursiva ou utilizando uma Pilha (LIFO). Uma característica crucial da DFS é a classificação temporal dos eventos. Cormen et al. (2009) sugerem registrar dois carimbos de tempo para cada vértice u:

- $d[u]$: Momento de descoberta (quando o vértice passa de branco para cinza).
- $f[u]$: Momento de finalização (quando o vértice passa de cinza para preto, após toda sua lista de adjacência ser explorada).

Classificação de arestas: a execução da DFS induz uma estrutura de floresta (Floresta DFS) e permite classificar as arestas do grafo original em quatro tipos, fundamentais para entender ciclos e estrutura (CORMEN et al., 2009)

- **Arestas de Árvore:** Arestas (u, v) percorridas pela busca quando v é descoberto.
- **Arestas de Retorno (Back edges):** Conectam um vértice u a um ancestral v na árvore DFS. A existência destas arestas indica a presença de ciclos.
- **Arestas de Avanço (Forward edges):** Conectam u a um descendente v que não é seu filho direto.
- **Arestas de Cruzamento (Cross edges):** Conectam vértices sem relação de ancestralidade.

A complexidade da DFS, assim como a BFS, é $\theta(V + E)$ para grafos representados por listas de adjacência.

2.3 Caminhos mais curtos

O problema de encontrar o caminho mais curto entre dois vértices em um grafo é um dos problemas de otimização combinatória mais estudados, servindo como sub-rotina para diversas aplicações, incluindo roteamento de redes e algoritmos de fluxo.

Seja $G = (V, E)$ um grafo ponderado com uma função de peso $w : E \to \mathbb{R}$. O peso de um caminho $p = \langle v_0, v_1 \ldots, v_k \rangle$ é a soma dos pesos de suas arestas constituintes: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$. O **caminho mais curto** de um vértice u a um vértice v é definido como qualquer caminho p com peso mínimo $\delta(u, v)$ (CORMEN et al., 2009).

2.3.1 Princípio do relaxamento e subestrutura ótima

Os algoritmos de caminho mínimo baseiam-se na propriedade de **subestrutura ótima**: subcaminhos de um caminho mais curto são, por si mesmos, caminhos mais curtos. Além disso, utilizam a técnica de **relaxamento** (KLEINBERG; TARDOS, 2005).

Para cada vértice v, o algoritmo mantém um atributo $d[v]$, que é um limite superior para o peso do caminho mais curto da fonte s até v. O processo de relaxar uma aresta (u, v) consiste em testar se é possível melhorar o caminho para v passando por u:

$$\text{Se } d[v] > d[u] + w(u, v), \text{ então } d[v] \leftarrow d[u] + w(u, v) \tag{2}$$

Algorithm 1: Método de relaxamento (CORMEN et al., 2009)

```
(1) Procedure Relax(u, v, w)
(2)     if v.d > u.d + w(u, v) then
(3)         v.d ← u.d + w(u, v)
(4)         v.π ← u
(5)     end
```

A diferença entre os algoritmos reside na ordem e na frequência com que as arestas são relaxadas.

2.3.2 Algoritmo de Dijkstra

Para grafos onde os pesos das arestas são não-negativos ($w(u, v) \geq 0$), o Algoritmo de Dijkstra é a abordagem mais eficiente. Ele utiliza uma estratégia gulosa, mantendo um conjunto de vértices cujos pesos finais dos caminhos mais curtos já foram determinados (AHO; HOPCROFT; ULLMAN, 1974).

O algoritmo seleciona repetidamente o vértice u com a menor estimativa de caminho mais curto $d[u]$ de uma fila de prioridade, adiciona-o ao conjunto de visitados e relaxa todas as arestas que saem de u. Dasgupta e Vazirani (2006) destacam que a eficiência do Dijkstra depende da estrutura de dados utilizada. Com um Heap Binário, a complexidade é $O((V + E) \log V)$. Cormen et al. (2009) apontam que, ao utilizar um Heap de Fibonacci, a complexidade amortizada pode ser reduzida para $O(E + VlogV)$, o que é crucial para grafos densos.

Algorithm 2: Algoritmo de Dijkstra (CORMEN et al., 2009)

```
(1)  Procedure Dijkstra(G, w, s)
(2)      Initialize-Single-Source(G, s)
(3)      S ← ∅
(4)      Q ← G.V
(5)      while Q ≠ ∅ do
(6)          u ← Extract-Min(Q)
(7)          S ← S ∪ {u}
(8)          foreach vertex v ∈ G.Adj[u] do
(9)              Relax(u, v, w)
(10)         end
(11)     end
```

2.3.3 Algoritmo de Bellman-Ford

Quando o grafo contém arestas com pesos negativos, o algoritmo de Dijkstra não garante a corretude. Nesses casos, utiliza-se o Algoritmo de Bellman-Ford. Segundo Manber (1989), este algoritmo baseia-se em programação dinâmica, relaxando todas as arestas do grafo $|V| - 1$ vezes.

A propriedade fundamental do Bellman-Ford é sua capacidade de detectar ciclos negativos. Se, após $|V| - 1$ iterações, ainda for possível relaxar alguma aresta, o grafo contém um ciclo com peso total negativo acessível a partir da fonte, o que implica que $\delta(s, v) = -\infty$ para alguns vértices (KLEINBERG; TARDOS, 2005). Sua complexidade é $O(VE)$.

Algorithm 3: Algoritmo de Bellman-Ford (CORMEN et al., 2009)

```
(1)  Procedure Bellman-Ford(G, w, s)
(2)      Initialize-Single-Source(G, s)
(3)      for i = 1 to |G.V| − 1 do
(4)          foreach edge (u, v) ∈ G.E do
(5)              Relax(u, v, w)
(6)          end
(7)      end
(8)      foreach edge (u, v) ∈ G.E do
(9)          if v.d > u.d + w(u, v) then
(10)             return false
(11)         end
(12)     end
(13)     return true
```

2.3.4 Algoritmo de Floyd-Warshall

Enquanto Dijkstra e Bellman-Ford resolvem o problema de fonte única (Single-Source), o algoritmo de Floyd-Warshall resolve o problema de caminhos mais curtos entre todos os pares de vértices (All-Pairs).

Aho, Hopcroft e Ullman (1974) explicam que o algoritmo utiliza programação dinâmica baseada na numeração dos vértices. Seja $d_{ij}^{(k)}$ o peso do caminho mais curto de i a j utilizando apenas vértices do conjunto $1, 2, \ldots, k$ como intermediários. A recorrência é dada por:

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) \tag{3}$$

A complexidade temporal é $\theta(V^3)$. Jungnickel (2008) observa que, para grafos densos, este método é geralmente mais eficiente do que executar o algoritmo de Bellman-Ford a partir de cada vértice.

Algorithm 4: Algoritmo de Floyd-Warshall (CORMEN et al., 2009)

Procedure *Floyd-Warshall(W)*
  $n \leftarrow W.rows$
  $D^{(0)} \leftarrow W$
  for $k = 1$ **to** n **do**
    let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix
    for $i = 1$ **to** n **do**
      for $j = 1$ **to** n **do**
        $d_{ij}^{(k)} \leftarrow \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right)$
      end
    end
  end
  return $D^{(n)}$

2.4 Redes de fluxo em grafos

A teoria dos fluxos em redes é um ramo fundamental da otimização combinatória que modela problemas de transporte de recursos através de um sistema conectado. Esta seção define os conceitos preliminares e apresenta os teoremas centrais que sustentam os algoritmos utilizados neste trabalho.

2.4.1 Definições básicas

Uma rede de fluxo é definida por um grafo direcionado $G = (V, E)$, onde V é o conjunto de vértices e E é o conjunto de arestas. Distinguem-se dois vértices especiais: a **fonte** $s \in V$. que produz o fluxo, e o **sorvedouro** $t \in V$, que o consome. Para cada aresta $(u, v) \in E$, associa-se uma capacidade não-negativa $c(u, v) \geq 0$, que limita a quantidade de fluxo que pode atravessar aquela aresta. Um **fluxo** em G é uma função $f : E \to \mathbb{R}^+$ que satisfaz as seguintes propriedades (CORMEN et al., 2009):

- **Restrição de capacidade:** O fluxo em uma aresta não pode exceder a sua capacidade.

$$0 \leq f(u, v) \leq c(u, v), \qquad \forall (u, v) \in E \tag{4}$$

- **Conservação de fluxo:** para todo vértice $v \in V - \{s, t\}$, a quantidade total de fluxo que entra deve ser igual à que sai

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) \tag{5}$$

2.4.2 O problema do fluxo máximo

O **problema do fluxo máximo** consiste em encontrar um fluxo f tal que $|f|$ seja o maior possível. Para entender a limitação desse fluxo, precisamos entender os conceitos de corte, redes residuais e caminhos aumentantes.

2.4.2.1 Corte

Um **corte** (S, T) é uma partição do conjunto de vértices V em dois subconjuntos disjuntos S e T, tal que $s \in S$ e $t \in T$. A **capacidade do corte**, denotada por $c(S, T)$, é a soma das capacidades das arestas que partem de S para T (CORMEN et al., 2009):

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v) \tag{6}$$

O **corte mínimo** de uma rede N é um corte cuja capacidade é a menor entre todos os cortes possíveis nessa rede (CORMEN et al., 2009). Isso nos leva ao **teorema do fluxo máximo e corte mínimo (Max-flow Min-cut)**

> *O valor máximo do fluxo de uma rede N é igual à capacidade do corte mínimo em N (JUNGNICKEL, 2008)*

Este teorema é fundamental para a compreensão e corretude dos algoritmos de caminhos aumentantes.

2.4.2.2 Redes residuais

A maioria dos algoritmos de resolução de problemas de fluxo máximo se baseia no conceito de redes residuais. Uma **rede residual** G_f consiste em arestas com capacidades que representam como podemos alterar o fluxo das arestas em G. A capacidade das arestas da rede residual G_f corresponde à diferença entre a capacidade da aresta original de G e o fluxo f que está passando pela aresta (CORMEN et al., 2009). Ou seja:

$$c_f(u, v) = c(u, v) - f(u, v) \tag{7}$$

Quando a aresta (u, v) possui fluxo igual à sua capacidade, então $c_f(u, v) = 0$ e a aresta não é incluída no grafo residual G_f

2.4.2.3 Caminhos aumentantes

Dado um grafo $G = (V, E)$ e um fluxo f, um **caminho aumentante** é um caminho de s para t na rede residual G_f. Por definição, podemos aumentar o fluxo em uma aresta (u, v) de um caminho aumentante em até $c_f(u, v)$ sem violar a capacidade da aresta original em G (CORMEN et al., 2009).

O método de Ford-Fulkerson, que é base dos principais algoritmos de resolução do problema do fluxo máximo, iterativamente encontra um caminho aumentante, calcula sua capacidade residual e adiciona esse fluxo à solução atual até que não existam mais caminhos aumentantes (AHUJA; MAGNANTI; ORLIN, 1993).

Algorithm 5: Algoritmo de Ford-Fulkerson

```
(1)  Procedure FORD-FULKERSON(G, s, t)
(2)      foreach edge (u, v) ∈ G.E do
(3)          (u, v).f ← 0
(4)      end
(5)      while there exists a path p from s to t in the residual network G_f do
(6)          c_f(p) ← min{c_f(u, v) : (u, v) is in p}
(7)          foreach edge (u, v) ∈ p do
(8)              if (u, v) ∈ E then
(9)                  (u, v).f ← (u, v).f + c_f(p)
(10)             else
(11)                 (v, u).f ← (v, u).f − c_f(p)
(12)             end
(13)         end
(14)     end
```

2.4.3 Fluxo de custo mínimo

Uma outra variação dos problemas em redes de fluxo em grafos é o problema do fluxo de custo mínimo. Essa variação considera, além das capacidades disponíveis, uma dimensão econômica. Seja $G = (V, E)$ uma rede direcionada onde cada aresta (u, v) possui uma capacidade $c(u, v)$ e um custo unitário $w(u, v)$. O objetivo é transmitir uma quantidade de fluxo pré-determinada F da fonte s ao sorvedouro t com o menor custo total possível.

A formulação linear do problema é dada por Ahuja, Magnanti e Orlin (1993). O primeiro objetivo é minimizar o custo total do fluxo, que segue a fórmula:

$$\text{Minimizar o custo do fluxo:} \quad z(f) = \sum_{(u,v)\in A} w(u,v) \cdot f(u,v) \tag{8}$$

A função a seguir assegura que o nó fonte s gere exatamente a demanda F, e que o nó sorvedouro t absorva-a integralmente.

$$\sum_{i,j\in V} f(i,j) - \sum_{k,i\in V} f(k,i) = \begin{cases} F & \text{se } i = s \\ -F & \text{se } j = t \\ 0 & \text{caso contrário} \end{cases} \tag{9}$$

Além desta, é importante lembrar da equação 4, que define que o fluxo em uma aresta não pode exceder a sua capacidade.

2.4.3.1 Condições de otimalidade e ciclos negativos

O critério fundamental para verificar se uma solução é ótima é o **teorema dos ciclos negativos**:

> Um fluxo f é uma solução viável para o problema do fluxo de custo mínimo se e somente se ele satisfaz a condição de otimalidade de ciclo negativo: a rede residual $G(f)$ não contém um ciclo de custo negativo (AHUJA; MAGNANTI; ORLIN, 1993)

Este teorema é a base dos algoritmos de **cancelamento de ciclos** (Cycle Canceling), que iterativamente identificam ciclos negativos na rede residual e enviam fluxo através deles para reduzir o custo total da função objetivo até que nenhum ciclo negativo reste.

2.4.3.2 Potenciais de vértices e custos reduzidos

Embora o cancelamento de ciclos seja teoricamente sólido, a detecção de ciclos negativos (via Bellman-Ford) pode ser computacionalmente custosa. Para permitir o uso de algoritmos mais eficientes como o de Dijkstra, a literatura introduz o conceito de **potenciais de vértices**, baseando-se na teoria de dualidade da programação linear.

A ideia é manter um **potencial** para cada vértice v do grafo, chamado de $h(v)$, com valor inicial 0. Definimos o **custo reduzido** de uma aresta (u, v) como $\gamma'(u, v) = \gamma(u, v) + h(u) - h(v)$. Se os potenciais forem escolhidos de forma adequada, todos os custos reduzidos podem ser garantidos como não-negativos, permitindo o uso do algoritmo de Dijkstra. Além disso, é garantido que os caminhos de custo mínimo no grafo original e no grafo com custos reduzidos são os mesmos. (JUNGNICKEL, 2008).

O conceito de potenciais e de custo reduzido será utilizado em algoritmos como o de caminhos sucessivos para resolução do problema de atribuição na seção 3.2.3.3.

2.5 Problema Geral do Emparelhamento

O problema de **matching** (emparelhamento) em grafos é um problema fundamental na otimização combinatória

Um **matching** M em um grafo não-direcionado $G = (V, E)$ é um subconjunto de arestas $M \subseteq E$ tal que nenhum par de arestas em M compartilha um vértice comum (KLEINBERG; TARDOS, 2005). Em outras palavras, cada nó aparece em no máximo uma aresta de M.

- Um vértice é chamado **coberto** (*matched*) se for incidente a uma aresta em M; caso contrário, é **descoberto** (*unmatched* ou *exposed*) (CORMEN et al., 2009)
- Um **matching de cardinalidade máxima** (Maximum Matching) é um matching com o maior número possível de arestas (CORMEN et al., 2009). A cardinalidade máxima de um matching é denotada por $v(G)$ (SCHRIJVER, 2004)

- Um **matching perfeito** é um matching que cobre todos os vértices do grafo (KLEINBERG; TARDOS, 2005)
- O **problema de matching ponderado** (Weighted Matching Problem) envolve encontrar um matching para o qual a soma dos pesos das arestas é máxima. Em um grafo ponderado $G = (V, E, w)$, busca-se um $M \subseteq E$ que maximize $w(M)$ (MANDULAK et al., 2024)

2.5.1 Matching em grafos bipartidos

O problema de **Matching Bipartido** é o caso clássico de encontrar um matching de cardinalidade máxima em um grafo bipartido $G = (V, E)$, onde $V = X \cup Y$ (KLEINBERG; TARDOS, 2005)

- O matching em grafos bipartidos pode modelar situações de atribuição, como associar empregos (X) a máquinas (Y), ou professores (X) a cursos (Y), onde uma aresta indica uma capacidade de atribuição (KLEINBERG; TARDOS, 2005)
- O problema de matching ponderado em grafos bipartidos é equivalente ao **problema de atribuição (LAWLER, 1976)**, que historicamente motivou o desenvolvimento do **método Húngaro** (SCHRIJVER, 2004)

2.5.2 Formulação geral (emparelhamento e otimização)

Em um contexto mais amplo, o emparelhamento de grafos pode ser formalizado como um problema de otimização que busca maximizar a compatibilidade entre dois grafos G e G' (CAETANO et al., 2009).

- O problema de graph matching é frequentemente abordado como um **problema de atribuição quadrática (QAP)**. Essa formulação busca maximizar uma função objetivo que combina termos de compatibilidade unária (nó-nó, $c_{ii'}$) e compatibilidade par a par (aresta-aresta, $d_{ii'jj'}$), sujeito a restrições de atribuição binária ($y_{ii'} \in 0, 1$). O termo quadrático codifica a preservação das relações (arestas) entre os nós (CAETANO et al., 2009)
- Para grafos bipartidos, a determinação de um matching máximo pode ser resolvida de maneira eficiente e está intimamente ligada a problemas de *network flow* (fluxo em redes) (KLEINBERG; TARDOS, 2005)

3 Problemas de emparelhamento em grafos bipartidos

3.1 Emparelhamento de cardinalidade máxima

3.1.1 Descrição do problema

Dado um grafo não-direcionado $G = (V, E)$, o problema de emparelhamento de cardinalidade máxima (MCM) busca encontrar um emparelhamento $M \subseteq E$ tal que o número de arestas em M seja maximizado, ou seja, $|M|$ é o maior entre todos os emparelhamentos possíveis em G.

Um caso especial, mas importante, do MCM é o **Emparelhamento perfeito**, que é um emparelhamento onde todos os vértices do grafo são cobertos por exatamente uma aresta do emparelhamento. Para que um emparelhamento perfeito exista, o grafo deve ter um número par de vértices e o emparelhamento máximo deve ter cardinalidade igual a $|V|/2$ (JUNGNICKEL, 2008).

3.1.2 Propriedades

Antes de continuar para os algoritmos e soluções, é necessário entender algumas propriedades importantes

3.1.2.1 Emparelhamento máximo vs maximal

Um emparelhamento máximo é aquele que contém o maior número possível de arestas, enquanto um emparelhamento maximal é aquele que não pode ser aumentado adicionando mais arestas, mas não necessariamente é o maior possível (MANBER, 1989).

3.1.2.2 Caminhos aumentantes

Um caminho aumentante P em um grafo G é chamado de caminho aumentante se P tem tamanho ímpar, começa e termina em vértices livres (não cobertos por nenhuma aresta do emparelhamento atual) e alterna entre arestas que pertencem a M e arestas que não pertencem a M. A existência de um caminho aumentante indica que o emparelhamento atual não é máximo (SCHRIJVER, 2004)

3.1.2.3 Teorema de Kőnig

O teorema de Kőnig estabelece uma relação fundamental entre emparelhamentos e coberturas em grafos bipartidos. Ele afirma que, em qualquer grafo bipartido, o tamanho do maior emparelhamento é igual ao tamanho da menor cobertura de vértices (JUNGNICKEL, 2008).

A cobertura de vértices é um conjunto de vértices tal que cada aresta do grafo é incidente a pelo menos um vértice desse conjunto. O teorema de Kőnig é uma ferramenta poderosa para resolver problemas de emparelhamento, utilizado nos algoritmos de Hopcroft-Karp e no método Húngaro. A seguir será detalhado um método algorítmico para encontrar a cobertura de vértices mínima a partir de um emparelhamento máximo.

O método é definido pelos seguintes passos (JUNGNICKEL, 2008):

1. Identifica-se o conjunto de vértices livres (não saturados por M) na partição esquerda U.
2. A partir desses vértices, inicia-se uma travessia no grafo (utilizando Busca em Largura ou Profundidade) com regras estritas de alternância: transições do conjunto U para V devem ocorrer exclusivamente através de arestas que **não** pertencem ao emparelhamento ($E \setminus M$), enquanto as transições de V de volta para U devem utilizar obrigatoriamente as arestas do emparelhamento (M).
3. Registra-se em um conjunto Z todos os vértices que foram alcançados durante essa busca, tanto da partição U quanto da partição V.
4. A Cobertura Mínima de Vértices C é construída selecionando-se os vértices de U que **não** foram visitados e os vértices de V que **foram** visitados. Matematicamente, a cobertura é dada por $C = (U \setminus Z) \cup (V \cap Z)$.

Essa extração algorítmica possui complexidade linear em relação ao tamanho do grafo, $O(V + E)$, visto que requer apenas uma travessia simples após o cálculo do emparelhamento. Sua implementação é uma sub-rotina fundamental para métodos exatos de problemas mais complexos, como a atualização de limiares no Método Dual para o problema de atribuição gargalo (CORMEN et al., 2009).

3.1.2.4 Teorema de Hall

O Teorema de Hall, frequentemente referido como Teorema do Casamento de Hall, é outro resultado estrutural para grafos bipartidos. Enquanto o teorema de König determina a cardinalidade do emparelhamento máximo, o teorema de Hall estabelece a condição necessária e suficiente para a existência de um emparelhamento que cubra integralmente um dos subconjuntos da partição (SCHRIJVER, 2004).

Para um grafo bipartido $G = (U \cup V, E)$, o teorema afirma que existe um emparelhamento cobrindo todo o conjunto U se, e somente se, para qualquer subconjunto $S \subseteq U$, o tamanho de sua vizinhança $N(S)$ (o conjunto de vértices em V adjacentes a pelo menos um vértice em S) for maior ou igual ao tamanho de S. Matematicamente, exige-se que $|N(S)| \geq |S|$ para todo $S \subseteq U$ (JUNGNICKEL, 2008). Essa propriedade, conhecida como Condição de Hall, é essencial para provar matematicamente a viabilidade de atribuições e identificar gargalos estruturais que impedem a formação de um emparelhamento perfeito, relacionando-se intimamente com os conceitos de cortes mínimos em redes de fluxo.

3.1.2.5 Teorema de Gallai e Conjunto Independente Máximo

Um conjunto independente em um grafo é um subconjunto de vértices no qual não existem duas extremidades adjacentes, ou seja, nenhuma aresta conecta dois vértices desse conjunto. O problema de encontrar o Conjunto Independente Máximo (*Maximum Independent Set* - MIS) é classicamente conhecido por pertencer à classe NP-Difícil para grafos gerais (GAREY; JOHNSON, 1990). No entanto, quando o grafo é restrito à classe dos grafos bipartidos, o problema pode ser resolvido em tempo polinomial graças à sua profunda relação com os emparelhamentos e coberturas.

A base dessa resolução é o Teorema de Gallai, proposto em 1959. O teorema estabelece que, para qualquer grafo $G = (V, E)$ que não possua vértices isolados, a soma do tamanho da Cobertura Mínima de Vértices (MVC) com o tamanho do Conjunto Independente Máximo (MIS) é sempre igual ao número total de vértices do grafo (SCHRIJVER, 2004).

Matematicamente:

$$|MVC| + |MIS| = |V| \tag{10}$$

Pelo Teorema de König, detalhado anteriormente, sabemos que em grafos bipartidos o tamanho da Cobertura Mínima de Vértices é exatamente igual ao tamanho do Emparelhamento Máximo ($|MVC| = |M|$). Consequentemente, o tamanho do Conjunto Independente Máximo pode ser deduzido diretamente do emparelhamento: $|MIS| = |V| - |M|$ (JUNGNICKEL, 2008).

Além da equivalência de cardinalidade, o Teorema de Gallai também fornece a dualidade estrutural: o Conjunto Independente Máximo é exatamente o complemento da Cobertura Mínima de Vértices. Portanto, para construir algoritmicamente o Conjunto Independente Máximo em um grafo bipartido, basta executar o seguinte procedimento:

1. Calcula-se o Emparelhamento Máximo utilizando algoritmos como o de Hopcroft-Karp.
2. Extrai-se a Cobertura Mínima de Vértices C através do método de busca por caminhos alternantes a partir dos vértices livres, definido na seção 3.1.2.3.
3. O Conjunto Independente Máximo I será composto por todos os vértices do grafo que não pertencem à cobertura. Ou seja, $I = V \setminus C$.

3.1.3 Algoritmos exatos

Por ser em um ambiente mais restrito, o problema de emparelhamento de cardinalidade máxima tem diversas soluções em tempo polinomial. A seguir listadas algumas delas:

3.1.3.1 Algoritmo do caminho aumentante

O algoritmo de caminho aumentante se baseia em um lema proposto por Claude Berge, que afirma que um grafo G é máximo se e somente se não existe nenhum caminho aumentante em G. Este algoritmo é uma implementação direta deste lema, onde se busca repetidamente por caminhos aumentantes e se aumenta o emparelhamento até que nenhum caminho aumentante possa ser encontrado (LAWLER, 1976).

Começamos com um emparelhamento vazio $M = \emptyset$. Escolhemos um vértice livre u no lado esquerdo do grafo bipartido e tentamos encontrar um caminho aumentante P que começa em u e termina em um vértice livre no lado direito do grafo, respeitando a regra da alternância entre arestas em M e arestas fora de M. Se um caminho aumentante for encontrado, atualizamos o emparelhamento M invertendo as arestas ao longo do caminho P. Repetimos esse processo até que não seja possível encontrar mais caminhos aumentantes. O emparelhamento resultante será o emparelhamento de cardinalidade máxima (HALIM; HALIM, 2013).

Em seguida o algoritmo em pseudo-código:

Algorithm 6: Algoritmo de caminhos aumentantes (MCBM) (HALIM; HALIM, 2013)

```
(1)  Global: match (vetor de emparelhamento), vis (vetor de visitados), AdjList

(2)  Procedure Aug(l)
(3)      if vis[l] = 1 then
(4)          return 0
(5)      end
(6)      vis[l] ← 1
(7)      for each r ∈ AdjList[l] do
(8)          if match[r] = −1 or Aug(match[r]) then
(9)              match[r] ← l
(10)             return 1 [comment: found 1 matching]
(11)         end
(12)     end
(13)     return 0 [comment: no matching]

(14) Procedure MCBM_Algorithm(n)
(15)     MCBM ← 0
(16)     match ← −1 (initialize for all V)
(17)     for l ← 0 to n − 1 do
(18)         vis ← 0 (reset before each recursion)
(19)         MCBM ← MCBM + Aug(l)
(20)     end
(21)     return MCBM
```

Como o algoritmo repete a busca por caminhos aumentantes para cada vértice do lado esquerdo do grafo, o tempo de execução total do algoritmo é $O(VE)$, onde V é o número de vértices e E é o número de arestas no grafo bipartido (HALIM; HALIM, 2013).

3.1.3.2 Redução ao problema de fluxo máximo

O problema de emparelhamento de cardinalidade máxima em grafos bipartidos pode ser eficientemente resolvido através de uma redução ao problema de fluxo máximo em redes. A ideia central é construir uma rede de fluxo a partir do grafo bipartido original, onde cada aresta do grafo bipartido é convertida em uma aresta com capacidade unitária na rede de fluxo. (LAWLER, 1976)

Primeiro, adicionamos um vértice fonte s e um vértice sumidouro t à rede. Conectamos o vértice fonte s a todos os vértices do conjunto esquerdo do grafo bipartido com arestas de capacidade 1. Em seguida, conectamos todos os vértices do conjunto direito do grafo bipartido ao vértice sumidouro t, também com arestas de capacidade 1. As arestas entre os conjuntos esquerdo e direito do grafo bipartido são mantidas na rede de fluxo, cada uma com capacidade 1 (HALIM; HALIM, 2013).

Com isso, temos agora um grafo de fluxo onde o objetivo é encontrar o fluxo máximo do vértice fonte s para o vértice sumidouro t. O valor do fluxo máximo encontrado nesta rede corresponde ao tamanho do emparelhamento máximo no grafo bipartido original. (LAWLER, 1976). Para encontrar o fluxo máximo, podemos utilizar algoritmos clássicos como o de Edmonds-Karp.

3.1.3.3 Algoritmo de Edmonds-Karp

O algoritmo de **Edmonds-Karp** é o principal método para resolver esse tipo de problema. Ele é uma especialização do método de Ford-Fulkerson descrito na seção 2.4.2.3 que utiliza uma busca em largura (BFS) para encontrar sempre o caminho aumentante mais curto em número de arestas. Tem complexidade de tempo $O(VE^2)$:

Algorithm 7: Algoritmo de Edmonds-Karp (HALIM; HALIM, 2013)

```
(1)  Global: res (matriz de capacidade residual), mf, f, s, t, p (vetor de pais da BFS)

(2)  Procedure BFS(s, t)
(3)      dist ← ∞;    dist[s] ← 0
(4)      Q ← {s};    p ← nil (init vector with -1)
(5)      while Q ≠ ∅ do
(6)          u ← Q.front(); Q.pop()
(7)          if u = t then
(8)              break (stop BFS)
(9)          end
(10)         for each v ∈ V do
(11)             if res_{u,v} > 0 and dist[v] = ∞ then
(12)                 dist[v] ← dist[u] + 1
(13)                 Q.push(v)
(14)                 p[v] ← u
(15)             end
(16)         end
(17)     end

(18) Procedure Augment(v, minEdge)
(19)     if v = s then
(20)         f ← minEdge
(21)         return
(22)     else
(23)         if p[v] ≠ −1 then
(24)             Augment(p[v], min(minEdge, res_{p[v],v}))
(25)             res_{p[v],v} ← res_{p[v],v} − f
(26)             res_{v,p[v]} ← res_{v,p[v]} + f
(27)         end
(28)     end

(29) Procedure EdmondsKarp(s, t)
(30)     mf ← 0
(31)     while true do
(32)         f ← 0
(33)         BFS(s, t); [comment: fills the parent vector p]
(34)         Augment(t, ∞); [comment: find min edge weight f in path]
(35)         if f = 0 then
(36)             break (terminate)
(37)         end
(38)         mf ← mf + f
(39)     end
(40)     return mf
```

O pseudocódigo divide-se em três procedimentos:

- **BFS:** Busca em largura a partir de s na rede residual, percorrendo apenas arestas com capacidade residual positiva. Ao alcançar cada vértice pela primeira vez, registra seu pai em p; como a busca é em largura, o caminho registrado até t é o de menor número de arestas.
- **Augment:** Percorre o caminho encontrado de t até s, seguindo o vetor de pais. Na volta da recursão, f guarda a menor capacidade residual do caminho (o gargalo), que é subtraída das arestas no sentido do caminho e somada às arestas reversas.

- **EdmondsKarp:** Repete a busca e o aumento enquanto existir caminho de s a t. Quando a BFS não alcança t, nenhum fluxo é enviado ($f = 0$) e o fluxo acumulado mf é máximo.

3.1.3.4 Algoritmo de Dinic

Outra opção para resolver o problema de fluxo máximo é o algoritmo de Dinic, que é mais eficiente em muitos casos práticos, especialmente em grafos densos. O algoritmo de Dinic utiliza uma combinação de buscas em largura (BFS) para construir níveis na rede residual e buscas em profundidade (DFS) para encontrar caminhos aumentantes dentro desses níveis. O tempo de execução do algoritmo de Dinic é $O(V^2E)$ em redes gerais e cai para $O(E\sqrt{V})$ em redes de capacidade unitária, como a obtida na redução do emparelhamento bipartido, o que o torna uma escolha preferida para muitos problemas de fluxo máximo (HALIM; HALIM, 2013).

Este trabalho não tem uma implementação do algoritmo de Dinic. Ao invés disso, será apresentado o algoritmo de Hopcroft-Karp, que é uma variação especializada do algoritmo de Dinic para o problema de emparelhamento em grafos bipartidos, oferecendo uma solução mais eficiente para este caso específico.

3.1.3.5 Algoritmo de Hopcroft-Karp

O algoritmo de Hopcroft-Karp é um algoritmo eficiente para encontrar o emparelhamento máximo em grafos bipartidos. Ele melhora o desempenho do algoritmo de caminhos aumentantes ao encontrar múltiplos caminhos aumentantes em cada iteração, em vez de apenas um. O tempo de execução do algoritmo de Hopcroft-Karp é $O(E\sqrt{V})$, tornando-o significativamente mais rápido do que o algoritmo de caminhos aumentantes simples, especialmente em grafos grandes (HALIM; HALIM, 2013).

O algoritmo consiste em duas fases principais: a fase de construção de níveis e a fase de busca de caminhos aumentantes. Na fase de construção de níveis, uma busca em largura (BFS) é realizada a partir dos vértices livres no lado esquerdo do grafo bipartido para construir uma camada de níveis que ajuda a identificar os caminhos aumentantes mais curtos. Na fase de busca de caminhos aumentantes, uma busca em profundidade (DFS) é realizada para encontrar todos os caminhos aumentantes possíveis dentro da camada de níveis construída na fase anterior. Esses caminhos são então usados para aumentar o emparelhamento (HALIM; HALIM, 2013).

A seguir, o pseudo-código do algoritmo de Hopcroft-Karp:

Algorithm 8: Algoritmo de Hopcroft-Karp (BURKARD; DELL'AMICO; MARTELLO, 2009)

```
(1)  let G = (U, V; E) be a bipartite graph with initial (possibly empty) matching M;
(2)  let U_0 contain all unmatched vertices of U;
(3)  let V_0 contain all unmatched vertices of V;
(4)  while U_0 ≠ ∅ do
(5)  |  Construct_layered_graph;
(6)  |  Find_set_ΔM;
(7)  |  M := M ⊖ ΔM;
(8)  |  update sets U_0, V_0 of unmatched vertices
(9)  end

(10) Procedure Construct_layered_graph
(11) |  L_0 := U_0, k* := k := 0;
(12) |  while L_k ≠ ∅ do
(13) |  |  for each i ∈ L_k do
(14) |  |  |  N(i) := {j : [i, j] ∈ E \ M, j ∉ L_1 ∪ L_3 ∪ ··· ∪ L_{k−1}};
(15) |  |  end
(16) |  |  L_{k+1} := ⋃_{i∈L_k} N(i);
(17) |  |  if L_{k+1} = ∅ then
(18) |  |  |  stop [comment: M is a maximum matching]
(19) |  |  end
(20) |  |  if L_{k+1} ∩ V_0 ≠ ∅ then
(21) |  |  |  k* := k + 1;
(22) |  |  |  L_{k+2} := ∅
(23) |  |  else
(24) |  |  |  L_{k+2} := {ī : [ī, j] ∈ M, j ∈ L_{k+1}};
(25) |  |  end
(26) |  |  k := k + 2
(27) |  end

(28) Procedure Find_set_ΔM
(29) |  [comment: find a maximal set ΔM of disjoint augmenting paths]
(30) |  delete all vertices in L_{k*} \ V_0;
(31) |  mark all remaining vertices as unscanned;
(32) |  k := 1; [comment: path counter]
(33) |  while L_0 ≠ ∅ do
(34) |  |  choose x_0 := i ∈ L_0 and delete it in L_0;
(35) |  |  ℓ := 0;
(36) |  |  while ℓ ≥ 0 do
(37) |  |  |  while x_ℓ has an unscanned neighbor in L_{ℓ+1} do
(38) |  |  |  |  choose unscanned neighbor x_{ℓ+1};
(39) |  |  |  |  mark x_{ℓ+1} as scanned;
(40) |  |  |  |  ℓ := ℓ + 1;
(41) |  |  |  |  if ℓ = k* then
(42) |  |  |  |  |  P_k := (x_0, x_1, ..., x_{k*});
(43) |  |  |  |  |  k := k + 1
(44) |  |  |  |  end
(45) |  |  |  end
(46) |  |  |  if ℓ < k* then
(47) |  |  |  |  ℓ := ℓ − 1
(48) |  |  |  else
(49) |  |  |  |  ℓ := −1
(50) |  |  |  end
(51) |  |  end
(52) |  end
(53) |  return ΔM := (P_1, P_2, ..., P_{k−1})
```

O pseudocódigo é organizado em um laço principal e dois procedimentos, explicados a seguir com referência às linhas do Algoritmo 8:

- **Laço principal (linhas 2–8):** U_0 e V_0 guardam os vértices livres de cada lado (linhas 2 e 3). Cada iteração do laço da linha 4 é uma *fase*: constrói-se o grafo em camadas (linha 5), encontra-se um conjunto maximal ΔM de caminhos aumentantes mínimos e disjuntos (linha 6) e o emparelhamento é atualizado pela diferença simétrica $M \ominus \Delta M$ (linha 7). Ao longo de cada caminho, as arestas emparelhadas e as não emparelhadas trocam de papel, de modo que $|M|$ aumenta em $|\Delta M|$.
- **Construção do grafo em camadas:** É uma busca em largura que parte de todos os vértices livres de U ao mesmo tempo, que formam a camada L_0 (linha 11). As camadas de índice ímpar contêm vértices de V, alcançados por arestas *não* emparelhadas (linha 14); a restrição $j \notin L_1 \cup \cdots \cup L_{k-1}$ impede que um vértice seja revisitado. Se nenhum vértice novo é alcançado (linha 17), não há caminho aumentante e, pelo lema de Berge, M é máximo. Se a nova camada contém um vértice livre de V (linha 20), foi encontrado o comprimento k^* dos caminhos aumentantes mais curtos, e a construção termina. Caso contrário, a camada seguinte é formada pelos parceiros, em M, dos vértices recém-alcançados (linha 24): o caminho segue obrigatoriamente pela aresta emparelhada, o que garante a alternância.
- **Busca dos caminhos disjuntos:** Primeiro, descartam-se os vértices da última camada que não são livres (linha 30), pois nenhum caminho aumentante termina neles. Em seguida, uma busca em profundidade parte de cada vértice livre $x_0 \in L_0$ (linha 34) e avança sempre para um vizinho ainda não visitado da camada seguinte (linha 37). Ao atingir a camada k^*, o caminho $P_k = (x_0, \ldots, x_{k^*})$ é registrado (linha 41) e a busca a partir dessa raiz termina; se a busca fica sem vizinhos antes disso, recua uma camada (linha 46). Como cada vértice é visitado no máximo uma vez por fase, os caminhos encontrados são disjuntos em vértices, e o conjunto ΔM é maximal.
- **Complexidade:** Cada fase examina cada aresta um número constante de vezes, com custo $O(E)$. Após $\sqrt{V}$ fases, todo caminho aumentante restante tem mais de $\sqrt{V}$ arestas; como os caminhos aumentantes de $M \ominus M^*$ são disjuntos em vértices, restam menos de $\sqrt{V}$ deles, e cada fase seguinte elimina ao menos um. O número de fases é, portanto, $O(\sqrt{V})$, o que leva à complexidade total $O(E\sqrt{V})$ (BURKARD; DELL'AMICO; MARTELLO, 2009).

3.1.4 Heurísticas

Por mais que o algoritmo de Hopcroft-Karp descrito em 8 seja considerado relativamente eficiente, com complexidade $O(E\sqrt{V})$, ele ainda não é tão viável para instâncias muito grandes do problema de emparelhamento de cardinalidade máxima. Por isso, foram propostos alguns métodos aproximados (heurísticas) com o objetivo de resolver o problema com complexidade linear $O(E)$.

3.1.4.1 Heurística gulosa simples

Um método que itera sobre as arestas do grafo em ordem arbitrária. Se a aresta (u, v) conecta dois vértices livres, adicione essa aresta ao emparelhamento e remova os vértices u e v. Tem complexidade de tempo linear $O(E)$ e garante um emparelhamento maximal, que conforme visto na seção 3.1.2.1, é um emparelhamento ao qual não é possível adicionar mais arestas, mas que não é necessariamente o de maior cardinalidade. A solução também garante um emparelhamento de pelo menos $1/2$ (metade) da cardinalidade máxima (VAZIRANI, 2001).

Algorithm 9: Heurística gulosa simples (VAZIRANI, 2001)

Input: bipartite graph $G = (U, V; E)$
Output: maximal matching M

(1) $M := \emptyset$
(2) **for** *each edge* $(u, v) \in E$*, in arbitrary order* **do**
(3) | **if** u *and* v *are both unmatched* **then**
(4) | | $M := M \cup \{(u, v)\}$
(5) | | mark u and v as matched
(6) | **end**
(7) **end**
(8) **return** M

3.1.4.2 Heurística de Karp-Sipser

Heurística descrita por Karp e Sipser (1981) para grafos esparsos que toma decisões com base na topologia do grafo ao invés de decisões aleatórias, tornando essas decisões mais seguras.

O algoritmo funciona reduzindo dinamicamente o grafo $G = (V, E)$ através da aplicação repetida de duas regras, nesta ordem de prioridade:

1. **Redução de folhas de grau 1:** Se existir algum vértice v no grafo atual com **grau 1** (apenas um vizinho u), a aresta (u, v) é selecionada imediatamente para o emparelhamento. Isso é seguro pois, se u é o único vizinho de v, ele também é a única opção para emparelhar v. Se isso não for feito, v ficará isolado.

2. **Escolha Aleatória:** Se **não** existirem vértices de grau 1 (ou seja, todos os vértices têm grau ≥ 2), o algoritmo escolhe uma aresta (u, v) aleatoriamente, adiciona ao emparelhamento e remove u e v do grafo.

   Em grafos esparsos, isso tende a quebrar ciclos e formar novas folhas (vértices de grau 1), permitindo que o algoritmo volte a usar a regra 1. Porém, em grafos densos, essa escolha aleatória pode comprometer a precisão da solução.

Para grafos esparsos, a regra 2 é raramente aplicada, pois a remoção de vértices na regra 1 frequentemente cria novas folhas, provocando um efeito cascata, fazendo com que a maior parte do grafo seja resolvida de maneira ótima. Com estruturas de dados adequadas, a complexidade do algoritmo é linear ($O(E)$). A implementação avaliada neste trabalho, por simplicidade, localiza os vértices de grau 1 por varredura, o que eleva sua complexidade a $O(V \cdot E)$.

Algorithm 10: Heurística de Karp-Sipser (KARP; SIPSER, 1981)

Input: bipartite graph $G = (U, V; E)$
Output: maximal matching M

(1) $M := \emptyset$
(2) **while** $E \neq \emptyset$ **do**
(3) | **if** *there is a vertex* v *with* $\deg(v) = 1$ **then**
(4) | | let u be the only neighbor of v
(5) | | **[comment: safe: some maximum matching contains** (u, v)**]**
(6) | **else**
(7) | | choose an arbitrary edge $(u, v) \in E$
(8) | | **[comment: the only step that may lose optimality]**
(9) | **end**
(10) | $M := M \cup \{(u, v)\}$
(11) | remove u, v and all their incident edges from G
(12) **end**
(13) **return** M

Tabela 1: Algoritmos para emparelhamento de cardinalidade máxima em grafos bipartidos

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Caminhos aumentantes	$O(VE)$	Exato	Sim	Busca iterativa (DFS/BFS) por caminhos alternantes entre vértices livres, invertendo arestas para aumentar a cardinalidade.
Edmonds-Karp	$O(VE^2)$	Exato	Sim	Implementação de Ford-Fulkerson que usa BFS para encontrar sempre o caminho aumentante mais curto (em nº de arestas) na rede residual.
Dinic	$O(V^2E)$	Exato	Não	Algoritmo de fluxo que constrói grafos de níveis (BFS) e satura múltiplos caminhos simultaneamente via fluxo de bloqueio (DFS). Em redes de capacidade unitária, como a do emparelhamento bipartido, cai para $O(E\sqrt{V})$.
Hopcroft-Karp	$O(E\sqrt{V})$	Exato	Sim	Busca múltiplos caminhos aumentantes disjuntos por fase usando BFS e DFS.
Gulosa simples	$O(E)$	Heurística	Sim	Itera sobre arestas arbitrariamente, selecionando aquelas que conectam dois vértices livres.
Karp-Sipser	$O(E)$	Heurística	Sim*	Prioriza remoção de vértices de grau 1; usa escolha aleatória quando não há folhas.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 9. * A implementação avaliada localiza os vértices de grau 1 por varredura simples, com complexidade $O(V \cdot E)$.

3.2 Problema de atribuição (Assignment Problem)

3.2.1 Descrição do problema

O problema de atribuição, também conhecido como problema do emparelhamento ponderado, consiste em encontrar uma combinação ótima de atribuições entre dois conjuntos disjuntos, minimizando o custo total associado a essas atribuições. Exemplo: Considere N trabalhadores e N tarefas, onde cada trabalhador pode ser designado a exatamente uma tarefa, e cada tarefa deve ser atribuída a exatamente um trabalhador. O custo de atribuir o trabalhador i à tarefa j é representado por uma matriz de custos $W = [w_{ij}]$ (LAWLER, 1976). O objetivo é encontrar um conjunto de atribuições que minimize o custo total.

3.2.2 Propriedades

3.2.3 Algoritmos exatos

3.2.3.1 Método Húngaro

O método Húngaro é um algoritmo eficiente para resolver o problema de atribuição em tempo polinomial. Ele foi desenvolvido por Harold Kuhn em 1955 e é baseado no trabalho de Dénes Kőnig e Jenő Egerváry sobre emparelhamentos em grafos bipartidos (JUNGNICKEL, 2008).

O método se baseia em manter um conjunto de potenciais para os vértices dos dois conjuntos do grafo bipartido, sendo u_i o potencial do vértice i no conjunto esquerdo e v_j o potencial do vértice j no conjunto direito. A regra que o algoritmo mantém é que para cada aresta (i, j), a soma dos potenciais deve ser maior ou igual ao custo da aresta, ou seja, $u_i + v_j \geq w_{ij}$.

Cria-se um subgrafo de igualdade $H_{u,v}$ contendo apenas as arestas onde a soma dos potenciais é igual ao custo da aresta, ou seja, $u_i + v_j = w_{ij}$. O algoritmo então tenta encontrar um emparelhamento máximo neste subgrafo de igualdade. Se o emparelhamento encontrado cobre todos os vértices do conjunto esquerdo, então ele é ótimo para o problema de atribuição original. Caso contrário, o algoritmo ajusta os potenciais para criar novas arestas de igualdade e repete o processo até encontrar um emparelhamento máximo que cubra todos os vértices do conjunto esquerdo (JUNGNICKEL, 2008).

Em seguida o pseudo-código do método Húngaro:

Algorithm 11: Algoritmo Húngaro (JUNGNICKEL, 2008)

```
(1)  Procedure HUNGARIAN(n, w; mate)
(2)      for v ∈ V do mate(v) ← 0
(3)      for i = 1 to n do u_i ← max{w_ij : j = 1, ..., n}; v_i ← 0
(4)      nrex ← n
(5)      while nrex ≠ 0 do
(6)          for i = 1 to n do m(i) ← false; p(i) ← 0; δ_i ← ∞
(7)          aug ← false; Q ← {i ∈ S : mate(i) = 0}
(8)          repeat
(9)              remove an arbitrary vertex i from Q; m(i) ← true; j ← 1
(10)             while aug = false and j ≤ n do
(11)                 if mate(i) ≠ j' then
(12)                     if u_i + v_j − w_ij < δ_j then
(13)                         δ_j ← u_i + v_j − w_ij; p(j) ← i
(14)                         if δ_j = 0 then
(15)                             if mate(j') = 0 then
(16)                                 AUGMENT(mate, p, j'; mate)
(17)                                 aug ← true; nrex ← nrex − 1
(18)                             else
(19)                                 Q ← Q ∪ {mate(j')}
(20)                             end
(21)                         end
(22)                     end
(23)                 end
(24)                 j ← j + 1
(25)             end
(26)             if aug = false and Q = ∅ then
(27)                 J ← {i ∈ S : m(i) = true}; K ← {j' ∈ T : δ_j = 0}
(28)                 δ ← min{δ_j : j' ∈ T \ K}
(29)                 for i ∈ J do u_i ← u_i − δ
(30)                 for j' ∈ K do v_j ← v_j + δ
(31)                 for j' ∈ T \ K do δ_j ← δ_j − δ
(32)                 X ← {j' ∈ T \ K : δ_j = 0}
(33)                 if mate(j') ≠ 0 for all j' ∈ X then
(34)                     for j' ∈ X do Q ← Q ∪ {mate(j')}
(35)                 else
(36)                     choose j' ∈ X with mate(j') = 0
(37)                     AUGMENT(mate, p, j'; mate)
(38)                     aug ← true; nrex ← nrex − 1
(39)                 end
(40)             end
(41)         until aug = true
(42)     end
```

Algorithm 12: Método AUGMENT (JUNGNICKEL, 2008)

```
(1)  Procedure AUGMENT(mate, p, j'; mate)
(2)      repeat
(3)          i ← p(j); mate(j') ← i; next ← mate(i); mate(i) ← j'
(4)          if next ≠ 0 then
(5)              j' ← next
(6)          end
(7)      until next = 0
```

Na versão de Jungnickel (2008), o algoritmo *maximiza* o peso do emparelhamento; para minimizar custos, basta usar $w_{ij} = -c_{ij}$. As linhas da matriz formam o conjunto $S = \{1, \ldots, n\}$ e as colunas, o conjunto $T = \{1', \ldots, n'\}$, em que j' denota a coluna j como vértice do grafo. A seguir, cada parte do Algoritmo 11 é explicada com referência às suas linhas:

- **Inicialização (linhas 2–4):** Nenhum vértice começa emparelhado. Cada potencial de linha u_i recebe o maior peso da linha i, e os potenciais de coluna v_j recebem zero, o que satisfaz $u_i + v_j \geq w_{ij}$ para toda aresta. O contador $nrex$ guarda o número de linhas ainda livres.
- **Fases (linha 5):** Cada iteração do laço externo aumenta o emparelhamento em uma aresta, até que todas as linhas estejam emparelhadas. No início da fase, as marcas são reiniciadas (linha 6): $m(i)$ indica se a linha i já entrou na árvore de busca, $p(j)$ guarda a linha pela qual a coluna j foi alcançada e δ_j guarda a menor folga $u_i + v_j - w_{ij}$ da coluna j em relação às linhas da árvore. A fila Q começa com as linhas livres, que são as raízes da busca (linha 7).
- **Busca no subgrafo de igualdade:** Uma linha i é retirada da fila e entra na árvore (linha 9). Para cada coluna que não é a sua parceira atual (linha 11), calcula-se a folga da aresta (i, j) e, se ela for menor que δ_j, a coluna passa a ter i como predecessora (linha 12). Folga nula significa que a aresta pertence ao subgrafo de igualdade $H_{u,v}$ (linha 14). Nesse caso, se a coluna está livre, foi encontrado um caminho aumentante, e o procedimento AUGMENT o aplica (linha 16); senão, a parceira da coluna entra na fila (linha 19), e a árvore cresce pela aresta emparelhada.
- **Ajuste dos potenciais (linhas 26–36):** Se a fila esvazia sem que um caminho aumentante seja encontrado, as linhas e colunas da árvore formam os conjuntos J e K. Calcula-se δ, a menor folga entre as colunas fora da árvore (linha 28). Diminuir u_i em δ nas linhas da árvore e aumentar v_j em δ nas colunas da árvore preserva as arestas da árvore no subgrafo de igualdade e mantém a viabilidade dos potenciais. Ao mesmo tempo, a folga das colunas de fora diminui em δ (linha 31), o que coloca ao menos uma nova aresta no subgrafo de igualdade (o conjunto X, linha 32). Se todas as colunas de X estão emparelhadas, suas parceiras entram na fila e a busca continua; senão, uma coluna livre de X completa um caminho aumentante (linha 36).
- **Aumento (Algoritmo 12):** A partir da coluna livre encontrada, o procedimento AUGMENT segue os predecessores p de volta até a raiz, invertendo o emparelhamento ao longo do caminho.
- **Complexidade:** Em cada fase, cada linha entra na árvore no máximo uma vez, e sua varredura custa $O(n)$. Cada ajuste de potenciais custa $O(n)$ e acrescenta ao menos uma coluna à árvore, de modo que há no máximo n ajustes por fase. Cada fase custa, portanto, $O(n^2)$, e as n fases resultam em $O(n^3)$.

3.2.3.2 Jonker-Volgenant

O algoritmo de Jonker-Volgenant é uma evolução otimizada para a resolução do problema de atribuição linear, proposto por Roy Jonker e Anton Volgenant em 1987. Ele foi desenvolvido para superar o desempenho prático do método Húngaro, especialmente em grafos densos, combinando a teoria de dualidade com estratégias de inicialização e busca mais eficientes (JONKER; VOLGENANT, 1987).

Assim como o método Húngaro, este algoritmo baseia-se na manutenção de potenciais duais (u_i e v_j) e respeita as condições de folga complementar. No entanto, sua principal distinção conceitual é o uso da estratégia de Caminho Aumentante Mais Curto (Shortest Augmenting Path). Ao invés de construir emparelhamentos máximos em fases distintas, o algoritmo foca em encontrar o caminho de custo mínimo que conecta uma linha não atribuída a uma coluna livre.

O processo inicia com heurísticas avançadas (conhecidas como redução de colunas e transferência de redução) para resolver rapidamente as atribuições triviais. Para os vértices restantes, o algoritmo realiza uma busca similar ao algoritmo de Dijkstra: ele explora o grafo para encontrar o caminho aumentante mais curto, atualizando os potenciais duais simultaneamente durante a busca. Isso garante que, a cada iteração, o emparelhamento seja aumentado pelo menor custo possível até que a solução ótima completa seja atingida.

A implementação completa deste algoritmo é extensa; o Algoritmo 13 apresenta suas quatro fases em alto nível. Sua eficiência prática o torna uma escolha preferida para muitos problemas de atribuição em aplicações reais, especialmente quando comparado ao método Húngaro tradicional (JUNGNICKEL, 2008).

Algorithm 13: Algoritmo de Jonker-Volgenant (LAPJV), em alto nível (JONKER; VOLGENANT, 1987)

Input: cost matrix $C = (c_{ij})_{n \times n}$
Output: optimal assignment x (row i is assigned to column x_i) and column duals v
(1) **[comment: phase 1: column reduction]**
(2) **for** $j := n$ **to** 1 **do**
(3) | $i^* := \arg\min_i c_{ij}$; $v_j := c_{i^* j}$
(4) | **if** *row* i^* *is unassigned* **then** assign i^* to j
(5) **end**
(6) **[comment: phase 2: reduction transfer]**
(7) **for** *each assigned row* i *that is the minimum of a single column* $j_1 = x_i$ **do**
(8) | $v_{j_1} := v_{j_1} - \min_{j \neq j_1}(c_{ij} - v_j)$
(9) **end**
(10) **[comment: phase 3: augmenting row reduction (applied twice)]**
(11) **for** *each free row* i **do**
(12) | let j_1, j_2 be the columns with the smallest and second smallest values $u_1 \leq u_2$ of $c_{ij} - v_j$
(13) | **if** $u_1 < u_2$ **then**
(14) | | $v_{j_1} := v_{j_1} - (u_2 - u_1)$
(15) | **else**
(16) | | **if** j_1 *is assigned* **then** $j_1 := j_2$
(17) | **end**
(18) | assign i to j_1; the row previously assigned to j_1, if any, becomes free
(19) **end**
(20) **[comment: phase 4: augmentation by shortest paths]**
(21) **for** *each remaining free row* i **do**
(22) | find a shortest augmenting path from i to a free column with a Dijkstra-like search on the reduced costs $c_{kj} - v_j$
(23) | $v_j := v_j + d_j - \mu$ for every scanned column j, where d_j is its distance and μ the length of the path
(24) | augment the assignment along the path
(25) **end**
(26) **return** x, v

3.2.3.3 Redução para problema de fluxo de custo mínimo

Similarmente ao que foi feito para o problema de emparelhamento de cardinalidade máxima, o problema de atribuição pode ser resolvido através de uma redução ao problema de fluxo de custo mínimo. Assim como para o problema anterior, essa redução é feita criando dois novos vértices: um vértice fonte s e um vértice sorvedouro t. Em seguida, são adicionadas arestas do vértice fonte s para cada vértice de um dos conjuntos do grafo bipartido, com capacidade 1 e custo 0. Também são adicionadas arestas de cada vértice do outro conjunto para o vértice sorvedouro t, também com capacidade 1 e custo 0. Finalmente, as arestas do grafo bipartido original são adicionadas com capacidade 1 e custo igual ao custo de atribuição correspondente.

Para resolver problemas de fluxo podemos utilizar algoritmos como o de caminhos de custo mínimo sucessivos (Successive Shortest Paths) ou o algoritmo de cancelamento de ciclos (Cycle canceling) (SCHRIJVER, 2004).

O primeiro algoritmo, o de caminhos de custo mínimo sucessivos, foi inicialmente proposto por Jewell, Busacker e Gowen e é uma adaptação do algoritmo de Ford-Fulkerson para incorporar o custo das arestas. Em poucas palavras, ele encontra o caminho de s para t de custo mínimo, envia o máximo fluxo possível ao longo desse caminho, e repete esse processo até que não seja mais possível aumentar o fluxo sem violar as capacidades das arestas (SCHRIJVER, 2004).

Algorithm 14: Algoritmo de caminhos de custo mínimo sucessivos (JUNGNICKEL, 2008)

(1) **Procedure** *OPTFLOW(*$G, c, s, t, \gamma, v; f$, *sol)*
(2) **for** $e \in E$ **do** $f(e) \leftarrow 0$
(3) sol $\leftarrow$ true, val $\leftarrow 0$
(4) **while** *sol* $=$ *true* **and** *val* $< v$ **do**
(5) construct the auxiliary network $N' = (G', c', s, t)$ with cost function γ'
(6) **if** t *is not accessible from* s *in* G' **then**
(7) sol $\leftarrow$ false
(8) **else**
(9) determine a shortest path P from s to t in (G', γ')
(10) $\delta \leftarrow \min\{c'(e) : e \in P\}; \delta' \leftarrow \min(\delta, v - \text{val}); \text{val} \leftarrow \text{val} + \delta'$
(11) augment f along P by δ'
(12) **end**
(13) **end**

A complexidade do algoritmo é dependente do método utilizado para encontrar o caminho P. A implementação padrão utilizaria o algoritmo de Dijkstra, que tem uma complexidade de $O(E + V \log V)$ em uma implementação com filas de prioridade, onde V é o número de vértices e E o número de arestas. Porém, como o grafo residual pode conter arestas com custos negativos, essa implementação não funcionaria corretamente. Para contornar esse problema, temos duas opções principais:

- **Utilizar Bellman-Ford:** A solução mais simples e direta é utilizar, ao invés do algoritmo de Dijkstra, o algoritmo de Bellman-Ford para encontrar o caminho de custo mínimo P. O Bellman-Ford é mais lento, com uma complexidade de $O(VE)$, mas lida corretamente com arestas de custo negativo.
- **Utilizar a técnica de Potenciais:** Permite que utilizemos o algoritmo de Dijkstra mesmo na presença de arestas de custo negativo.

  A cada iteração do loop, encontra-se o caminho de custo mínimo P utilizando Dijkstra com os custos reduzidos. Após encontrar o caminho, os potenciais são atualizados para todos os vértices v alcançáveis a partir de s no grafo residual, ajustando-os conforme a distância mínima encontrada pelo Dijkstra, de acordo com a fórmula $h(v) \leftarrow h(v) + d(s, v)$, onde $d(s, v)$ é a distância mínima de s até v no grafo residual com custos reduzidos.

  Abaixo uma adaptação do algoritmo 14 utilizando a técnica de potenciais:

Algorithm 15: Adaptação do algoritmo 14 utilizando Dijkstra com técnica de potenciais

(1) **Procedure** *OPTFLOW_DIJKSTRA(*$G, c, s, t, \gamma, v; f$, *sol)*
(2) **for** $e \in E$ **do** $f(e) \leftarrow 0$
(3) **for** $x \in V$ **do** $\pi(x) \leftarrow 0$
(4) sol $\leftarrow$ true, val $\leftarrow 0$
(5) **while** *sol* $=$ *true* **and** *val* $< v$ **do**
(6) construct the auxiliary network $N' = (G', c', s, t)$
(7) **for** $e = (u, v) \in E(G')$ **do**
(8) define reduced cost: $w(e) \leftarrow \gamma(e) + \pi(u) - \pi(v)$
(9) **end**
(10) Run Dijkstra on N' using weights w to find shortest path distances $d(\cdot)$ from s
(11) **if** t *is not accessible from* s *(i.e.,* $d(t) = \infty$*)* **then**
(12) sol $\leftarrow$ false
(13) **else**
(14) **for** $x \in V$ *such that* $d(x) < \infty$ **do** $\pi(x) \leftarrow \pi(x) + d(x)$
(15) Identify shortest path P from s to t based on $d(\cdot)$
(16) $\delta \leftarrow \min\{c'(e) : e \in P\}; \delta' \leftarrow \min(\delta, v - \text{val}); \text{val} \leftarrow \text{val} + \delta'$
(17) augment f along P by δ'
(18) **end**
(19) **end**

A complexidade do algoritmo utilizando Dijkstra com a técnica de potenciais é $O(V(E + V \log V))$, onde V é o número de vértices e E o número de arestas do grafo. Isso ocorre porque, em cada iteração do loop principal, executamos o algoritmo de Dijkstra, que tem complexidade $O(E + V \log V)$, e o número máximo de iterações é limitado por V, o número de vértices no grafo bipartido.

3.2.4 Heurísticas

Assim como no problema de emparelhamento de cardinalidade máxima, as soluções exatas, apesar de apresentarem tempo polinomial, são inviáveis para instâncias muito grandes. A seguir algumas heurísticas que objetivam resolver o problema de atribuição em tempo até mesmo linear ($O(E)$).

3.2.4.1 Heurística gulosa ordenada

Similar à heurística gulosa simples descrita na seção 3.1.4.1, com a diferença de que a ordem das arestas não é mais arbitrária, e sim decrescente por peso. Tem a mesma garantia de metade do peso ótimo, mas tem complexidade $O(E \log E)$ por conta da ordenação.

Tabela 2: Algoritmos para problema de atribuição

Algoritmo	Complexidade	Tipo	Avaliado	Descrição
Método Húngaro	$O(n^3)$	Exato	Sim	Método primal-dual que mantém folga complementar para custo mínimo em bipartidos.
Jonker-Volgenant	$O(n^3)$	Exato	Sim	Algoritmo primal-dual eficiente que combina redução de custos inicial e busca de caminhos aumentantes com atualização otimizada de duais.
Custo mínimo (Bellman-Ford)	$O(V^2E)$	Exato	Sim	Método de caminhos sucessivos que usa Bellman-Ford para encontrar caminhos na rede residual, suportando custos negativos iniciais.
Custo mínimo (Dijkstra)	$O(V(E + V \log V))$	Exato	Não	Utiliza potenciais de vértices para manter custos reduzidos não-negativos, permitindo o uso do eficiente Dijkstra para aumentos sucessivos.
Gulosa ordenada	$O(E \log E)$	Heurística	Não	Ordena as arestas por peso decrescente e seleciona iterativamente as disponíveis.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 9.

3.3 Problema de atribuição gargalo

3.3.1 Descrição do problema

O problema de atribuição gargalo, também conhecido como **Problema de Emparelhamento Min-Max** ou **Bottleneck Assignment problem**, é uma variação do problema de atribuição tradicional. O objetivo deste problema é encontrar, em um grafo bipartido ponderado, um emparelhamento de cardinalidade máxima (maior número possível de arestas) no qual o mínimo dos pesos das arestas do emparelhamento seja maximizado (LAWLER, 1976).

Por exemplo. Considere N trabalhadores e N estações de trabalho. w_{ij} representa a eficiência do trabalhador i na estação de trabalho j. A eficiência total da produção é limitada pela eficiência do trabalhador menos eficiente. O objetivo é atribuir trabalhadores às estações de trabalho de forma a maximizar a eficiência mínima entre todas as atribuições. (LAWLER, 1976).

3.3.2 Soluções exatas

3.3.2.1 Métodos de threshold

Métodos de threshold funcionam alternando entre duas fases. A primeira fase consiste em selecionar um valor limite, chamado de **threshold**, descrito por $c*$. Na segunda fase, constrói-se um grafo bipartido com todas as arestas cujo peso é menor ou igual a $c*$. Em seguida, verifica-se se existe um emparelhamento

perfeito nesse grafo. O menor valor de $c*$ que permite um emparelhamento perfeito é a solução ótima para o problema de atribuição gargalo (BURKARD; DELL'AMICO; MARTELLO, 2009).

Para selecionar o valor limite $c*$, podemos utilizar uma busca binária entre o menor e o maior peso das arestas do grafo original. A cada iteração, ajustamos o valor de $c*$ com base na existência ou não de um emparelhamento perfeito no grafo construído. Esse processo continua até que o valor ótimo seja encontrado. Burkard, Dell'Amico e Martello (2009), que descreve esse método, utiliza um algoritmo baseado em matrizes para decidir se um emparelhamento é perfeito ou não, porém podemos utilizar qualquer algoritmo clássico de emparelhamento em grafos bipartidos, como os definidos na seção 3.1.3, como por exemplo o algoritmo de Hopcroft-Karp. Também segundo o autor, o algoritmo tem uma complexidade de tempo de $O(V^{2.5}/\sqrt{\log V})$.

Lawler (1976) menciona um método de threshold para solução do problema de atribuição gargalo, porém a definição de threshold dele é levemente diferente, referindo-se ao valor dinâmico do rótulo de um vértice durante busca de caminhos. O algoritmo que ele descreve se assemelha mais aos algoritmos de caminhos aumentantes, que serão discutidos na seção 3.3.2.3.

Algorithm 16: Algoritmo Threshold para o LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009)

(1) **let** $C = (c_{ij})$ be a given $n \times n$ cost matrix;
(2) $c_0^* := \min_{ij}\{c_{ij}\}, c_1^* := \max_{ij}\{c_{ij}\}$
(3) **if** $c_0^* = c_1^*$ **then**
(4) | $z := c_0^*$ **[comment: any permutation of** $\{1, 2, \ldots, n\}$ **is optimal]**
(5) **else**
(6) | **while** $C^* = \{c_{ij} : c_0^* < c_{ij} < c_1^*\} \neq \emptyset$ **do**
(7) | | **[comment: find the median,** c^***, of** C^***;]**
(8) | | $c^* := \min\{c \in C^* : |\{c_{ij} \in C^* : c_{ij} \leq c\}| \geq |C^*|/2\}$;
(9) | | Feasibility_check(c^*, c_0^*, c_1^*)
(10) | **end**
(11) | **if** *Feasibility_check has not yet been executed for* c_0^* **then**
(12) | | Feasibility_check(c_0^*, c_0^*, c_1^*)
(13) | **end**
(14) | find a perfect matching in the current bipartite graph $G[c_1^*]$
(15) | $z := c_1^*$ **[comment: value of the optimal matching]**
(16) **end**

(17) **Procedure** *Feasibility_check*(c^*, c_0^*, c_1^*)
(18) | define the current graph $G[c^*]$
(19) | **if** $G[c^*]$ *contains a perfect matching* **then**
(20) | | $c_1^* := c^*$
(21) | **else**
(22) | | $c_0^* := c^*$
(23) | **end**

3.3.2.2 Método dual

Assim como os métodos de threshold, o método dual também define um valor limite $c*$. No entanto, ao invés de 'chutar' um valor usando busca binária, este método define o valor inicial de $c*$ como sendo o menor valor possível que ele pode assumir. Olhando para a matriz de custos, esse valor é o máximo entre os mínimos de cada linha e cada coluna da matriz:

$$c* = \max\left\{\max_i(\min_j c_{ik}), \max_j(\min_i c_{kj})\right\} \tag{11}$$

Com o limite definido, verificamos se é possível encontrar um emparelhamento perfeito no grafo bipartido formado pelas arestas cujo peso é menor ou igual a $c*$, usando os mesmos algoritmos de antes. Se um emparelhamento perfeito for encontrado, então $c*$ é a solução ótima. Caso contrário, é necessário aumentar o valor de $c*$, e para isso nos baseamos no teorema de Kőnig, definido na seção 3.1.2.3.

Sabemos que todas as arestas $c_{ij} > c*$ estão conectadas a um conjunto de linhas I e um conjunto de colunas J. Essas linhas e colunas ”cobrem”todas as arestas do grafo atual. Para aumentar o valor de $c*$, precisamos adicionar uma nova aresta que não esteja coberta por esses conjuntos. Procuramos o menor peso entre as arestas não cobertas, ou seja, aquelas que não pertencem às linhas em I nem às colunas em J. Esse valor se torna o novo $c*$:

$$Novoc* = \min_{i \notin I, j \notin J} c_{ij} \tag{12}$$

A partir daqui, o processo se repete. Com o novo valor de $c*$ e a nova aresta adicionada ao grafo, verificamos novamente a existência de um emparelhamento perfeito. Esse ciclo continua até que um emparelhamento perfeito seja encontrado, momento em que o valor atual de $c*$ será a solução ótima para o problema de atribuição gargalo.

Algorithm 17: Dual_LBAP: algoritmo dual para o LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009)

```
(1)  let C = (c_ij) be a given n × n cost matrix;
(2)  c* := max_{k=1,2,...,n}(min_{i=1,2,...,n} c_ik, min_{j=1,2,...,n} c_kj)
(3)  M := ∅
(4)  while |M| < n do
(5)  |   define the bipartite graph G[c*]
(6)  |   find a maximum matching M in G[c*]
(7)  |   if |M| < n then
(8)  |   |   let I ⊆ U and J ⊆ V be vertex sets in a minimum vertex cover of G[c*]
(9)  |   |   c* := min_{i∉I, j∉J} c_ij
(10) |   end
(11) end
```

O método dual não foi implementado no ambiente de testes deste trabalho. Para o problema de atribuição gargalo, foram comparados o algoritmo Threshold (Algoritmo 16) e o método de caminhos aumentantes (Algoritmo 18), descrito a seguir.

3.3.2.3 Método de caminhos aumentantes

Ao contrário do método dual, que ajusta um limiar global para todo o grafo, o método de caminhos aumentantes é construtivo. Ele inicia com um emparelhamento vazio e busca aumentar sua cardinalidade iterativamente, adicionando um par de vértices de cada vez. O objetivo central em cada passo é encontrar um caminho aumentante P que conecte uma linha livre a uma coluna livre, de tal forma que a capacidade desse caminho seja mínima. A capacidade aqui é definida pelo peso da aresta mais cara contida no caminho:

$$cap(P) = \max_{(i,j) \in P} c_{ij} \tag{13}$$

Para encontrar tal caminho, utiliza-se uma adaptação do algoritmo de Dijkstra. Em vez de somar os pesos das arestas, mantemos para cada vértice um valor de rótulo que representa o ”gargalo”mínimo necessário para alcançá-lo. Durante a busca, ao tentar estender o caminho de um nó u (com rótulo d_u) para um vizinho v através da aresta (u, v), o custo não é somado, mas sim comparado. O novo custo potencial para atingir v é o máximo entre o gargalo já existente até u e o peso da nova aresta:

$$NovoCusto_v = \max(d_u, c_{uv}) \tag{14}$$

O algoritmo seleciona, a cada passo, o nó não visitado com o menor rótulo atual, garantindo uma exploração gulosa que minimiza o gargalo. Assim que uma coluna livre é alcançada por essa busca, o caminho aumentante é identificado e o emparelhamento é expandido, invertendo-se as arestas ao longo do caminho. Esse processo se repete (reiniciando as distâncias) até que um emparelhamento perfeito seja obtido. A solução do problema será determinada pela maior capacidade encontrada entre todos os caminhos aumentantes utilizados na construção.

Algorithm 18: LBAP usando caminhos aumentantes (BURKARD; DELL'AMICO; MARTELLO, 2009)

(1) **let** $G = (U, V; E)$ be a bipartite graph with edge lengths c_{ij} for $[i, j] \in E$;
(2) find a lower bound c^* for the optimum objective function value;
(3) **[comment: e.g.,** $c^* := \max(\max_{i \in U} \min_{j \in V} c_{ij},\ \max_{j \in V} \min_{i \in U} c_{ij})$**]**
(4) find a maximum matching M in the graph $G[c^*]$ having an edge $[i, j]$ **iff** $c_{ij} \leq c^*$;
(5) **if** $|M| = |U|$ **then**
(6) | **stop [comment:** M **is an optimum matching of cost** c^***]**
(7) **else**
(8) | **let** L be the set of unmatched vertices of U
(9) **end**
(10) **while** L *is nonempty* **do**
(11) | choose an arbitrary vertex $i \in L$;
(12) | $L := L \setminus \{i\}$;
(13) | Dijkstra(i); **[comment: the procedure returns a path** P **starting in** i**]**
(14) | **if** $P \neq \text{nil}$ **then**
(15) | | $M := M \ominus P$;
(16) | | $c^* := \max(c^*, \ell(P))$
(17) | **end**
(18) **end**
(19) **[comment:** M **is a maximum matching with minimum cost** c^***.]**

Algorithm 19: Dijkstra modificado para caminhos aumentantes em LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009)

(1) **Procedure** *Dijkstra(i)*
(2) label all vertices $j \in V$ by $(\alpha(j), \beta(j)) := (\infty, \text{nil})$;
(3) $R := V$; **[comment: R contains the unscanned vertices of V]**
(4) $\overline{\alpha}(i) := c^*, P := \text{nil}$;
(5) Label(i);
(6) **while** $R \neq \emptyset$ **do**
(7) find a vertex $j_1 \in R$ with minimum $\alpha(j_1)$;
(8) **if** $\alpha(j_1) = \infty$ **then**
(9) $R := \emptyset$
(10) **else**
(11) **if** j_1 *is unmatched* **then**
(12) find the path P induced by the vertex sequence $(i, \ldots, \overline{\beta}(\beta(j_1)), \beta(j_1), j_1)$;
(13) $\ell(P) := \alpha(j_1)$;
(14) $R := \emptyset$
(15) **else**
(16) let $[i_1, j_1]$ be the matching edge;
(17) label i_1 with $(\overline{\alpha}(i_1), \overline{\beta}(i_1)) = (\alpha(j_1), j_1)$;
(18) $R := R \setminus \{j_1\}$;
(19) Label(i_1);
(20) **end**
(21) **end**
(22) **end**

(23) **Procedure** *Label(i)*
(24) **for** *each neighbor* $j \in R$ *of* i **do**
(25) **if** $\alpha(j) > \max(\overline{\alpha}(i), c_{ij})$ **then**
(26) $\alpha(j) := \max(\overline{\alpha}(i), c_{ij})$;
(27) $\beta(j) := i$
(28) **end**
(29) **end**

3.3.3 Heurísticas

3.3.3.1 Heurística Construtiva Gulosa

Diferente do problema de atribuição, onde o guloso simples funciona bem, no problema de gargalo o guloso deve ser adaptado para evitar criar "bicos de garrafa"muito altos no final do processo.

O algoritmo seleciona arestas iterativamente. Para cada vértice livre, ele verifica não apenas a aresta de menor peso incidente, mas também o "custo de oportunidade"de não escolhê-la.

Se não escolhermos a aresta (u, v) de peso 10, a próxima melhor opção para u tem peso 100? Se sim, a prioridade de (u, v) aumenta drasticamente (MARTELLO; TOTH, 1987).

Tabela 3: Algoritmos para problema de atribuição gargalo

Algoritmo	Complexidade	Tipo	Avaliado	Descrição
Threshold	$O(V^{2.5}/\sqrt{\log V})$	Exato	Sim*	Aplica busca binária sobre os pesos das arestas, testando a existência de emparelhamento perfeito no subgrafo restrito ao limiar.
Método Dual	$O(K \cdot E\sqrt{V})$	Exato	Não	Expande um limiar inferior iterativamente baseado no Teorema de König, adicionando apenas arestas necessárias para cobrir componentes.
Caminhos aumentantes	$O(EV)$	Exato	Sim	Abordagem construtiva que busca caminhos aumentantes minimizando a aresta de peso máximo (gargalo) no trajeto (variação de Dijkstra).
Construtiva Gulosa	$O(E \log E)$	Heurística	Não	Constrói a solução selecionando arestas de baixo peso, usando critérios locais ou de "arrependimento"para evitar gargalos futuros.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 9. * A implementação avaliada testa cada limiar por caminhos aumentantes, com custo $O(n^3)$ por teste e $O(n^3 \log n)$ no total.

3.4 Problema do emparelhamento estável

3.4.1 Descrição do problema

O problema do emparelhamento estável, também conhecido como *Stable Marriage Problem* (SMP), é um problema clássico na teoria dos grafos e na ciência da computação, proposto inicialmente por Gale e Shapley em 1962 (KLEINBERG; TARDOS, 2005). Ele envolve a combinação de dois conjuntos distintos de elementos e, apesar de semelhante aos problemas de emparelhamento em grafos bipartidos estudados anteriormente, possui características e objetivos diferentes.

Imagine dois conjuntos distintos de mesmo tamanho, N homens e N mulheres. Cada indivíduo em ambos os conjuntos tem uma lista de preferências ordenadas para todos os membros do outro conjunto. O objetivo do SMP é encontrar um emparelhamento entre homens e mulheres de tal forma que não existam dois indivíduos, um homem e uma mulher, que prefeririam estar juntos em vez de com seus parceiros atuais. Se tal par existir, o emparelhamento é considerado instável.

O problema pode ser resolvido utilizando o algoritmo de Gale-Shapley, que usa um processo iterativo de propostas e rejeições para garantir que o emparelhamento final seja estável.

3.4.2 Propriedades

- **Estabilidade**: Um emparelhamento é estável se não houver dois indivíduos que prefeririam estar juntos em vez de com seus parceiros atuais. A estabilidade é a característica central do problema (LAWLER, 1976).
- **Existência de solução**: Gale e Shapley provaram que sempre existe pelo menos um emparelhamento estável para qualquer conjunto de preferências (AHUJA; MAGNANTI; ORLIN, 1993).
- **Otimalidade para quem propõe**: O emparelhamento resultante do algoritmo de Gale-Shapley é ótimo para o conjunto que faz as propostas (por exemplo, os homens), significando que cada indivíduo

nesse conjunto recebe o melhor parceiro possível dentro dos emparelhamentos estáveis (AHUJA; MAGNANTI; ORLIN, 1993).

- **Pessimismo para quem recebe propostas**: Por outro lado, o emparelhamento é o pior possível para o conjunto que recebe as propostas (por exemplo, as mulheres), significando que cada indivíduo nesse conjunto recebe o pior parceiro possível dentro dos emparelhamentos estáveis (AHUJA; MAGNANTI; ORLIN, 1993).
- **Não-unicidade**: Pode haver múltiplos emparelhamentos estáveis para um dado conjunto de preferências, dependendo das listas de preferências dos indivíduos (KLEINBERG; TARDOS, 2005).

3.4.3 Algoritmo de Gale-Shapley

A solução clássica para o problema do emparelhamento estável é o algoritmo de Gale-Shapley, também conhecido como o algoritmo de casamento estável. O algoritmo funciona sob a lógica de propostas e rejeições, onde um dos conjuntos (por exemplo, os homens) faz propostas às mulheres com base em suas listas de preferências.

O algoritmo pode ser descrito de forma simples da seguinte maneira:

Enquanto existir um homem h que ainda não propôs a todas as mulheres em sua lista:

1. O homem h propõe à primeira mulher m da sua lista de preferências a quem ele ainda não propôs.
2. A decisão da mulher m é tomada com base em suas preferências:
    - Se m não estiver comprometida, ela aceita a proposta de h. Eles ficam "noivos".
    - Se m já estiver comprometida com outro homem h', ela compara h e h'. Se m preferir h a h', ela rejeita h' (que fica livre) e aceita a proposta de h. Caso contrário, ela rejeita h (que continua livre e tenta a próxima da sua lista).

O algoritmo continua até que todos os homens estejam comprometidos. O resultado final é um emparelhamento estável, onde não existem pares instáveis (KLEINBERG; TARDOS, 2005).

Segue um exemplo em pseudocódigo do algoritmo:

Algorithm 20: Algoritmo de Gale-Shapley para o problema do emparelhamento estável (KLEINBERG; TARDOS, 2005)

```
(1)  Initially all m ∈ M and w ∈ W are free
(2)  while there is a man m who is free and hasn't proposed to every woman do
(3)      Choose such a man m
(4)      Let w be the highest-ranked woman in m's preference list to whom m has not yet proposed
(5)      if w is free then
(6)          (m, w) become engaged
(7)      else
(8)          w is currently engaged to m'
(9)          if w prefers m' to m then
(10)             m remains free
(11)         else
(12)             (m, w) become engaged
(13)             m' becomes free
(14)         end
(15)     end
(16) end
(17) Return the set S of engaged pairs
```

Neste trabalho, o algoritmo de Gale-Shapley foi avaliado em três implementações, cuja origem consta na Tabela 9.

4 Problemas de emparelhamento em grafos gerais

Ao contrário dos problemas em grafos bipartidos, problemas de emparelhamento em grafos gerais podem ser consideravelmente mais complexos. Alguns dos exemplos apresentados nesta seção são resolvíveis por algoritmo exatos em tempo polinomial, apesar de apresentarem uma complexidade de tempo considerada alta. Outros problemas pertencem à classe NP, o que significa que não foi proposto um algoritmo que pudesse resolver esses problemas em tempo poliomial.

4.1 Emparelhamento de cardinalidade máxima

4.1.1 Descrição

A definição do problema de emparelhamento de cardinalidade máxima é, fundamentalmente, a mesma da sua contraparte bipartida, descrita na seção 3.1. No entanto, a solução deste problema é significativamente mais complexa que a versão bipartida, mesmo ainda sendo em tempo polinomial.

4.1.2 Propriedades

4.1.2.1 Ciclos ímpares

Previamente, na seção 3.1.2.2, vimos a definição de caminhos aumentantes, que vieram a ser fundamentais para a solução de grande parte dos problemas de emparelhamento em grafos bipartidos. Em grafos gerais, a base da solução ainda é a mesma, se baseando no lema de Berge, descrito por Jungnickel (2008) como:

> Um emparelhamento M em um grafo G é máximo se, e somente se, não existe nenhum caminho aumentante em G relativo a M.

Em grafos bipartidos, ciclos sempre têm tamanho par. Isso garante que, ao realizar uma busca (BFS ou DFS) a partir de um vértice livre, nunca encontraremos uma aresta que conecte dois vértices que estejam à mesma distância (”paridade”) da raiz na árvore de busca alternante (AHUJA; MAGNANTI; ORLIN, 1993).

No entanto, em grafos gerais, a presença de ciclos de comprimento ímpar quebra essa lógica. Schrijver (2004) explica que, ao explorar o grafo, o algoritmo pode encontrar uma aresta conectando dois vértices que possuem paridade ”par”em relação à raiz da busca. Essa estrutura forma um **ciclo ímpar** que ”confunde”a identificação de um caminho aumentante simples, pois um vértice poderia ser alcançado por dois caminhos alternantes de paridades diferentes.

4.1.2.2 Flor (Blossom)

A solução para lidar com ciclos ímpares foi proposta por Edmonds (1965), e consiste em uma estrutura denominada **Blossom**.

Uma Blossom B é definida como um ciclo de comprimento ímpar $2k + 1$, onde k arestas pertencem ao emparelhamento M, de tal forma que existe apenas um vértice v_B no ciclo (a base da flor) que não é coberto por uma aresta de M dentro do próprio ciclo. A base v_B é conectada ao resto do grafo por um caminho alternante de comprimento par (o **caule** ou *stem*).

Edmonds provou que uma *blossom* pode ser contraída em um único vértice, com base no seguinte teorema:

> Existe um caminho aumentante no grafo original G se, e somente se, existe um caminho aumentante no grafo contraído $G' = G/B$, onde a flor B foi reduzida a um único vértice. (SCHRIJVER, 2004)

4.1.3 O algoritmo de Edmonds

Baseando-se na propriedade de Blossom e no teorema da contração, Lawler (1976) descreve o algoritmo para encontrar o emparelhamento máximo em grafos gerais da seguinte forma:

1. **Busca:** Inicie uma busca (geralmente BFS) a partir dos vértices livres para encontrar um caminho aumentante, construindo uma Floresta de Árvores Alternantes. Na versão de Jungnickel (2008) (Algoritmo 21), a busca parte de um vértice livre r de cada vez (linha 12) e cresce a árvore alternando uma aresta não emparelhada (x, y) com a aresta emparelhada $(y, mate(y))$ (linhas 33–35).

2. **Detecção:** Se a busca encontrar uma aresta que fecha um ciclo ímpar, uma **blossom** foi identificada. Isso ocorre quando a aresta (x, y) liga dois vértices de profundidade par da mesma árvore (linha 23). O procedimento BLOSSOM (Algoritmo 22) identifica o ciclo, subindo de x e de y até a raiz e tomando como base w o ancestral comum mais profundo.

3. **Contração:** Contraia todos os vértices da **blossom** em um único super-vértice e modifique o grafo temporariamente. O procedimento CONTRACT (Algoritmo 23) cria o super-vértice $b = n + m$, que herda o pai, a profundidade e o parceiro da base, recebe a união das adjacências da flor e desativa os vértices originais.

4. **Recursão:** Continue a busca no grafo contraído. Na versão de Jungnickel (2008), isso é feito sem recursão explícita: o super-vértice entra na fila de busca (linha 4 do Algoritmo 23) e passa a ser tratado como um vértice comum, podendo ele próprio fazer parte de novas flores.

5. **Expansão:** Se um caminho aumentante for encontrado no grafo contraído passando pelo super-vértice, ”abra”(expanda) a flor novamente para traduzir esse caminho para o grafo original. O caminho dentro da flor será roteado de forma a manter a alternância correta. É o primeiro laço do procedimento AUGMENT (Algoritmo 24, linha 6), que substitui cada super-vértice do caminho, do mais recente para o mais antigo, por um caminho alternante de comprimento par dentro da flor.

6. **Aumento:** Use o caminho encontrado para aumentar o emparelhamento através da diferença simétrica (inversão das arestas). O AUGMENT é chamado quando a busca alcança um vértice livre y (linha 30 do Algoritmo 21), e seu último laço (linha 16) inverte as arestas ao longo do caminho.

Schrijver (2004) detalha que, a depender da forma como o algoritmo é implementado, principalmente no que trata da contração, a complexidade do algoritmo ingênuo pode chegar a $O(V^3)$. No entanto, melhorias subsequentes nesse algoritmo culminaram em uma implementação em $O(E\sqrt{V})$, descrita por Micali e Vazirani (1980). No entanto, por conta da complexidade de implementação da solução mais rápida, a maioria das implementações práticas do algoritmo utilizam variantes com complexidade próxima a $O(V^3)$.

Os pseudocódigos a seguir, de Jungnickel (2008), organizam esses passos em quatro procedimentos: MAXMATCH, que conduz a busca; BLOSSOM, que identifica a flor; CONTRACT, que a contrai; e AUGMENT, que expande as flores e aumenta o emparelhamento. Cada passo acima indica a parte correspondente.

Algorithm 21: Procedimento MAXMATCH (JUNGNICKEL, 2008)

```
(1)  Procedure MAXMATCH(G; mate, nrex)
(2)      INMATCH(G; mate, nrex)
(3)      r ← 0
(4)      while nrex ≥ 2 and r < n do
(5)          r ← r + 1
(6)          if mate(r) = 0 then
(7)              Q ← ∅, aug ← false, m ← 0
(8)              for v ∈ V do
(9)                  p(v) ← 0, d(v) ← −1, a(v) ← true
(10)                 CA(v) ← A_v
(11)             end
(12)             d(r) ← 0; append r to Q
(13)             while aug = false and Q ≠ ∅ do
(14)                 remove the first vertex x of Q
(15)                 if a(x) = true then
(16)                     cont ← false
(17)                     for y ∈ CA(x) do
(18)                         u(y) ← false
(19)                     end
(20)                     repeat
(21)                         choose y ∈ CA(x) with u(y) = false; u(y) ← true
(22)                         if a(y) = true then
(23)                             if d(y) ≡ 0 (mod 2) then
(24)                                 m ← m + 1
(25)                                 BLOSSOM(x, y; B(m), w(m))
(26)                                 CONTRACT(B(m), m, w)
(27)                             end
(28)                             else if d(y) = −1 then
(29)                                 if mate(y) = 0 then
(30)                                     AUGMENT(x, y)
(31)                                 else
(32)                                     z ← mate(y)
(33)                                     p(y) ← x; d(y) ← d(x) + 1
(34)                                     p(z) ← y; d(z) ← d(y) + 1
(35)                                     insert z with priority d(z) into Q
(36)                                 end
(37)                             end
(38)                         end
(39)                     until u(y) = true for all y ∈ CA(x) or aug = true or cont = true
(40)                 end
(41)             end
(42)         end
(43)     end
```

Algorithm 22: Algoritmo de BLOSSOM (JUNGNICKEL, 2008)

```
(1)  Procedure BLOSSOM(x, y; B, w)
(2)      P ← {x}; P' ← {y}; u ← x; v ← y
(3)      repeat
(4)          P ← P ∪ {p(u)}; u ← p(u)
(5)      until p(u) = r
(6)      repeat
(7)          P' ← P' ∪ {p(v)}; v ← p(v)
(8)      until v = r
(9)      S ← P ∩ P'
(10)     let w be the element of S for which d(w) ≥ d(z) for all z ∈ S
(11)     B ← ((P ∪ P') \ S) ∪ {w}
```

Algorithm 23: Procedimento CONTRACT (JUNGNICKEL, 2008)

```
(1)  Procedure CONTRACT(B, m, w)
(2)      b ← n + m; a(b) ← true
(3)      p(b) ← p(w); d(b) ← d(w); mate(b) ← mate(w)
(4)      insert b into Q with priority d(b)
(5)      CA(b) ← ⋃_{z∈B} CA(z)
(6)      for z ∈ CA(b) do
(7)          CA(z) ← CA(z) ∪ {b}
(8)      end
(9)      for z ∈ B do
(10)         a(z) ← false
(11)     end
(12)     for z ∈ CA(b) do
(13)         if a(z) = true and p(z) ∈ B then
(14)             d(z) ← d(b) + 1; p(z) ← b
(15)             d(mate(z)) ← d(z) + 1
(16)         end
(17)     end
(18)     cont ← true
```

Algorithm 24: Procedimento AUGMENT (JUNGNICKEL, 2008)

(1) **Procedure** *AUGMENT*(x, y)
(2) $P \leftarrow \{y, x\}; v \leftarrow x$
(3) **while** $p(v) \neq 0$ **do**
(4) $P \leftarrow P \cup \{p(v)\}; v \leftarrow p(v)$
(5) **end**
(6) **while** *there exists* $b \in P$ *with* $b > n$ **do**
(7) choose the largest $b \in P$ with $b > n$
(8) $B \leftarrow B(b - n); w \leftarrow w(b - n); z \leftarrow mate(w)$
(9) let q be the neighbor of b on P which is different from z
(10) choose some $q' \in B \cap CA(q)$
(11) determine the alternating path B' of even length in B from w to q'
(12) replace b by w in P
(13) insert B' into P between w and q
(14) **end**
(15) $u \leftarrow y; v \leftarrow x$
(16) **while** $v \neq r$ **do**
(17) $z \leftarrow mate(v); mate(v) \leftarrow u; mate(u) \leftarrow v$
(18) $u \leftarrow z$; let v be the successor of z on P
(19) **end**
(20) $mate(v) \leftarrow u; mate(u) \leftarrow v$
(21) $nrex \leftarrow nrex - 2; aug \leftarrow$ **true**

4.1.3.1 Teorema de Tutte-Berge

O teorema de Tutte-Berge, apesar de não aparecer explicitamente na descrição do algoritmo, é fundamental para sua corretude, pois ele fornece um certificado de otimalidade. Por exemplo, digamos que a etapa de busca do algoritmo de Blossom não encontrou nenhum caminho aumentante. Como sabemos que o emparelhamento atual é máximo? Para descobrir isso, utilizamos a fórmula de Tutte-Berge, descrita por Jungnickel (2008) da seguinte forma:

$$v(G) = \frac{1}{2} \min_{U \subseteq V} (|V| + |U| - \mathrm{odd}(G - U)) \tag{15}$$

Onde $v(G)$ é o tamanho do emparelhamento máximo, e $\mathrm{odd}(G - U)$ é o número de componentes conexos com número ímpar de vértices no grafo resultante após a remoção de U. Se o grafo possui um emparelhamento perfeito, o mínimo desta equação é $|V|/2$, o que ocorre se e somente se $odd(G - U) \leq |U|$ para todo U.

4.1.4 Heurísticas

Algumas das heurísticas que resolvem este problema são as mesmas que vimos na seção 3.1, como a Heurística Gulosa Simples e a Heurística de Karp-Sipser, que também funcionam para grafos gerais. Mas, além dessas, existem outras heurísticas que podem ser utilizadas neste problema.

4.1.4.1 Algoritmo de crescimento de caminho (Path Growing)

Uma variação inteligente do método guloso, descrita por Drake e Hougardy (2003), que usa DFS e tenta evitar a ”miopia”de escolher uma aresta isolada.

1. Escolha um vértice livre inicial v.
2. Inicie uma busca em profundidade (DFS) para construir um caminho simples o mais longo possível.

3. Ao longo desse caminho, selecione arestas alternadas para o emparelhamento (a 1ª sim, a 2ª não, a 3ª sim...).

4. Remova os vértices usados e repita.

Ao contrário do guloso simples (que escolhe arestas desconexas), o Path Growing tende a cobrir cadeias longas de vértices, o que é estruturalmente vantajoso.

Tabela 4: Algoritmos para problema de emparelhamento de cardinalidade máxima em grafos gerais

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Edmonds	$O(V^3)$	Exato	Sim	Algoritmo exato (Blossom) que contrai ciclos ímpares temporariamente em super-vértices para identificar caminhos aumentantes.
Micali e Vazirani	$O(E\sqrt{V})$	Exato	Não	Variação otimizada do algoritmo de Edmonds que utiliza estruturas de dados avançadas para reduzir drasticamente a complexidade.
Gulosa simples	$O(E)$	Heurística	Não	Itera sobre arestas arbitrariamente, selecionando aquelas que conectam dois vértices livres.
Karp-Sipser	$O(E)$	Heurística	Não	Prioriza remoção de vértices de grau 1; usa escolha aleatória quando não há folhas.
Crescimento de caminho	$O(E)$	Heurística	Não	Heurística linear que constrói caminhos simples via DFS e seleciona arestas alternadas para garantir um emparelhamento maximal.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 9.

4.2 Emparelhamento ponderado máximo

Enquanto o problema de cardinalidade máxima busca o maior número de arestas, o problema do **Emparelhamento de Peso Máximo** (Maximum Weight Matching) introduz uma função de peso $w : E \to \mathbb{R}$ nas arestas, e o objetivo torna-se encontrar um emparelhamento M que maximize $\sum_{e \in M} w(e)$. Este problema é a generalização natural do Problema da Atribuição para grafos não-bipartidos, descrito em 3.2.

A complexidade deste problema em grafos gerais decorre, assim como no problema de emparelhamento de cardinalidade máxima descrito em 4.1, da presença de ciclos ímpares. Se tentarmos modelar o problema usando Programação Linear (PL) apenas com restrições de grau ($\sum x_e \leq 1$), a solução ótima do sistema relaxado pode ser fracionária (ex: $x_e = 0.5$ para as três arestas de um triângulo), o que não corresponde a um emparelhamento válido.

4.2.1 Formulação poliédrica e Restrições de Blossom

Conforme detalhado por Schrijver (2004), para descrever o **Poliedro do Emparelhamento** (Matching Polytope) em grafos gerais, Edmonds (1965) provou que é necessário adicionar uma família exponencial de restrições conhecidas como **Desigualdades de Blossom** (ou restrições de subconjuntos ímpares):

$$\sum_{e \in E(U)} x_e \leq \frac{|U| - 1}{2}, \quad \forall U \subseteq V, |U| \text{ ímpar}, |U| \geq 3 \tag{16}$$

Onde $E(U)$ é o conjunto de arestas com ambas as extremidades em U. Essas inequações garantem que, em qualquer subconjunto ímpar de vértices, o número de arestas selecionadas não exceda a capacidade máxima inteira possível.

4.2.2 A abordagem Primal-Dual

Diferente do caso de cardinalidade, onde buscamos apenas caminhos aumentantes, o algoritmo para o caso ponderado é um método **Primal-Dual**. Ele é uma generalização do Método Húngaro, mas adaptado para lidar com as blossoms.

Segundo Lawler (1976), o algoritmo mantém variáveis duais não apenas para os vértices (y_v), mas também para cada subconjunto ímpar de vértices (z_B) que forma uma blossom contraída. A condição de otimalidade baseia-se na folga complementar. Uma aresta (u, v) é considerada "admissível"ou "justa"(*tight*) se a soma dos duais correspondentes iguala o peso da aresta. Simplificadamente:

$$y_u + y_v + \sum_{B:(u,v)\in B} z_B = w_{uv} \tag{17}$$

O algoritmo opera iterativamente em dois estágios, descritos por Jungnickel (2008) e Schrijver (2004):

1. **Estágio Primal (Busca):** Restringindo-se apenas às arestas admissíveis (onde a equação acima é satisfeita), o algoritmo busca um caminho aumentante usando a técnica de contração de blossoms de Edmonds. Se encontrado, o emparelhamento é aumentado.
2. **Estágio Dual (Ajuste de Preços):** Se a busca falha em encontrar um caminho aumentante no grafo de arestas admissíveis, o algoritmo não termina (ao contrário do caso de cardinalidade). Em vez disso, ele realiza uma atualização das variáveis duais y e z.
    - O algoritmo calcula um valor δ (delta), que é a menor "folga"entre as arestas que conectam a árvore de busca atual a vértices externos.
    - Os duais dos vértices na árvore são ajustados por $\pm\delta$ e os duais das blossoms (z_B) também são ajustados.
    - Esse ajuste tem o efeito de "destravar"novas arestas (tornando-as admissíveis) ou permitir a expansão/contração de blossoms, possibilitando que a busca prossiga.

4.2.3 O algoritmo de Edmonds Ponderado

O procedimento resultante é frequentemente chamado de **Algoritmo de Blossom Ponderado** ou simplesmente Algoritmo de Edmonds para Emparelhamento Ponderado.

A implementação original de Edmonds era polinomial, $O(V^4)$. Lawler (1976) apresentou uma implementação $O(V^3)$, que por muito tempo foi o padrão em livros didáticos.

A variante $O(V^3)$ descrita por Lawler (1976) para o emparelhamento ponderado utiliza uma abordagem primal-dual rigorosa. O algoritmo mantém um conjunto de arestas admissíveis (custo reduzido nulo) baseadas em variáveis duais de vértices (y_i) e de subconjuntos ímpares (z_B). A cada iteração onde a busca por caminhos aumentantes falha, o algoritmo calcula um escalar crítico δ baseado nas folgas das arestas incidentes à árvore alternante atual (conjunto S). A atualização dual subsequente ($y_i \leftarrow y_i \pm \delta$, $z_B \leftarrow z_B \pm 2\delta$) garante o surgimento de novas arestas admissíveis ou a expansão de **blossoms** com dual nulo, mantendo a complexidade cúbica através do gerenciamento eficiente dos valores de folga mínima (**slack**) para vértices fora da árvore.

Lawler também destaca que a manipulação das variáveis duais das blossoms (z_B) é o que permite ao algoritmo "pagar"o custo de usar arestas dentro de estruturas cíclicas ímpares. Quando uma blossom é formada durante a busca, ela recebe um valor dual $z_B > 0$; quando ela precisa ser desfeita (expandida) para encontrar um caminho, seu valor dual retorna a zero.

O Algoritmo 25 resume esse esquema primal-dual em alto nível, omitindo as estruturas de dados de contração e expansão de blossoms. Nele, $\pi_{uv} = y_u + y_v + \sum_{B:(u,v)\in B} z_B - w_{uv}$ denota a folga da aresta (u, v); uma aresta é justa quando $\pi_{uv} = 0$.

Algorithm 25: Esquema primal-dual do algoritmo de Edmonds ponderado, em alto nível (LAWLER, 1976)

Input: graph $G = (V, E)$ with weights $w : E \to \mathbb{R}$
Output: maximum weight matching M

(1) $M := \emptyset$; $y_v := \frac{1}{2} \max_{e \in E} w(e)$ for all $v \in V$; $z_B := 0$ for every blossom B
(2) **while true do**
(3) | grow alternating trees rooted at the free vertices using only tight edges, labeling vertices as outer (S) or inner (T)
(4) | **if** *a tight edge joins outer vertices of two different trees* **then**
(5) | | augment M along the augmenting path found; remove all labels
(6) | **else if** *a tight edge joins two outer vertices of the same tree* **then**
(7) | | contract the blossom B formed by the edge, with $z_B := 0$
(8) | **else**
(9) | | $\delta_1 := \min\{y_v : v \text{ outer}\}$
(10) | | $\delta_2 := \min\{\pi_{uv} : u \text{ outer}, v \text{ unlabeled}\}$
(11) | | $\delta_3 := \min\{\pi_{uv}/2 : u, v \text{ outer, in different blossoms}\}$
(12) | | $\delta_4 := \min\{z_B/2 : B \text{ inner blossom}\}$
(13) | | $\delta := \min(\delta_1, \delta_2, \delta_3, \delta_4)$
(14) | | $y_v := y_v - \delta$ for outer v; $y_v := y_v + \delta$ for inner v
(15) | | $z_B := z_B + 2\delta$ for outer blossoms; $z_B := z_B - 2\delta$ for inner blossoms
(16) | | **if** $\delta = \delta_1$ **then stop [comment: M is optimal]**
(17) | | expand every inner blossom B with $z_B = 0$
(18) | **end**
(19) **end**
(20) **return** M, after expanding all remaining blossoms

4.2.4 Otimizações

Desenvolvimentos posteriores por Galil, Micali e Gabow (1986) e Gabow (1990) melhoraram a estrutura de dados para gerenciar a prioridade das atualizações duais e a contração de blossoms, atingindo complexidades próximas a $O(EV \log V)$ ou $O(E\sqrt{V}\alpha(E, V))$ em implementações de escalonamento (SCHRIJVER, 2004).

Apesar de ser polinomial (classe P), a implementação de emparelhamento ponderado em grafos gerais é considerada uma das rotinas mais complexas da otimização combinatória clássica, sendo significativamente mais difícil de implementar corretamente do que algoritmos para grafos bipartidos ou fluxo máximo.

4.2.5 Heurísticas

Aqui também podemos utilizar a mesma heurística que usamos para a versão do problema para grafos bipartidos: a Heurística Gulosa Ordenada, descrita na seção 3.2.4.1. Mas além dessas, temos:

4.2.5.1 Algoritmo de Crescimento de Caminho (Path Growing)

Similar à heurística descrita na seção 4.1.4.1, esta também faz uso de Path Growing para encontrar um emparelhamento. A vantagem desta é que não é necessário ordenação, tornando possível a execução em tempo linear ($O(E)$). Foi descrito por Drake e Hougardy (2003):

1. O algoritmo constrói caminhos simples disjuntos no grafo.
2. Comece em um vértice livre v qualquer. Estenda o caminho escolhendo a aresta de maior peso conectada a v que leve a um vértice ainda não visitado neste caminho.
3. Repita até que o caminho não possa crescer mais.

4. Dado um caminho $P = (e_1, e_2, e_3, \ldots, e_k)$, temos dois emparelhamentos possíveis formados por arestas alternadas:

5.
   - $M_1 = \{e_1, e_3, e_5, \ldots\}$ (Arestas ímpares)
   - $M_2 = \{e_2, e_4, e_6, \ldots\}$ (Arestas pares)

6. Compare o peso total de M_1 e M_2. Adicione o conjunto mais pesado à solução final e remova os vértices do grafo.

A diferença para a versão de emparelhamento de cardinalidade máxima é o passo 5, onde os pesos são somados para escolher o melhor conjunto. Garante emparelhamento de pelo menos $1/2$ do peso máximo (DRAKE; HOUGARDY, 2003).

O Algoritmo 26 corresponde à versão original de Drake e Hougardy (2003), que é a implementada neste trabalho. Nela, os conjuntos M_1 e M_2 são mantidos ao longo de todos os caminhos e o mais pesado é retornado ao final, em vez de a escolha ser feita caminho a caminho como na descrição acima; ambas as versões têm a mesma garantia de aproximação.

Algorithm 26: Algoritmo Path Growing (DRAKE; HOUGARDY, 2003)

Input: graph $G = (V, E)$ with weights $w : E \to \mathbb{R}$
Output: matching M with $w(M) \geq \frac{1}{2} w(M^*)$

(1) $M_1 := \emptyset$; $M_2 := \emptyset$; $i := 1$
(2) **while** $E \neq \emptyset$ **do**
(3) | choose a vertex x with $\deg(x) \geq 1$
(4) | **while** x *has a neighbor* **do**
(5) | | let (x, y) be the heaviest edge incident to x
(6) | | $M_i := M_i \cup \{(x, y)\}$; $i := 3 - i$
(7) | | remove x and all its incident edges from G
(8) | | $x := y$
(9) | **end**
(10) **end**
(11) **return** the heavier of M_1 and M_2

Tabela 5: Algoritmos para problema de emparelhamento ponderado máximo em grafos gerais

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Edmonds	$O(V^3)$	Exato	Sim	Método primal-dual que gerencia variáveis duais para vértices e blossoms, ajustando preços para encontrar caminhos no grafo de igualdade.
Edmonds otimizado	$O(EV \log V)$	Exato	Não	Melhoria do Edmonds original através de uma série de otimizações
Gulosa ordenada	$O(E \log E)$	Heurística	Não	Ordena as arestas por peso decrescente e seleciona iterativamente as disponíveis.
Crescimento de caminho	$O(E)$	Heurística	Sim	Heurística que estende caminhos pela aresta mais pesada local e seleciona o subconjunto de arestas (par ou ímpar) de maior soma total.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 9.

4.3 Emparelhamento induzido máximo

Diferentemente dos problemas de emparelhamento em grafos gerais vistos até o momento, que pertencem à classe de complexidade P, o problema do Emparelhamento Induzido Máximo (MIM - *Maximum Induced Matching*), frequentemente referido na literatura como **Emparelhamento Forte** (*Strong Matching*), pertence à classe dos problemas NP-difíceis. Apesar de ter uma variação estruturalmente simples quando comparado aos problemas clássicos, esse problema impõe uma restrição topológica severa sobre quais arestas podem coexistir na solução.

4.3.1 Propriedades

Seja $G = (V, E)$ um grafo geral. Um subconjunto de arestas $M \subseteq E$ é um **emparelhamento induzido** se satisfaz duas condições:

1. **Condição de Emparelhamento:** Nenhuma aresta em M compartilha um vértice (são disjuntas par a par).
2. **Condição Induzida:** O subgrafo induzido pelos vértices saturados por M, denotado por $G[V(M)]$, contém **apenas** as arestas de M.

Em termos mais simples, se tomarmos duas arestas quaisquer (u, v) e (x, y) pertencentes ao emparelhamento induzido, não pode existir no grafo original nenhuma aresta conectando um vértice do par $\{u, v\}$ a um vértice do par $\{x, y\}$. Isso implica que as arestas do emparelhamento devem estar "topologicamente isoladas"umas das outras.

Cameron (1989) oferece uma caracterização alternativa baseada em distância: um emparelhamento induzido é um conjunto de arestas onde a distância entre quaisquer duas arestas distintas é de pelo menos 2 (ou seja, não há aresta de ligação entre elas).

4.3.2 Complexidade computacional

Diferentemente do emparelhamento clássico (cardinalidade ou ponderado), que possui algoritmos polinomiais eficientes, o problema de encontrar o emparelhamento induzido de cardinalidade máxima é **NP-Difícil**.

Este resultado foi estabelecido no trabalho fundamental de Stockmeyer e Vazirani (1982). Eles demonstraram que o problema permanece NP-Difícil mesmo quando restrito a grafos bipartidos com grau máximo 3. Isso é um fato surpreendente e crucial: a propriedade de ser bipartido, que geralmente torna problemas de emparelhamento fáceis (como visto no Teorema de König), **não** ajuda no caso do emparelhamento induzido.

4.3.3 Redução ao Problema do Conjunto Independente

A intratabilidade do MIM pode ser explicada através de uma transformação de grafos. O problema de encontrar um emparelhamento induzido em G é equivalente a encontrar um **Conjunto Independente Máximo** (Maximum Independent Set - MIS) em um grafo transformado.

A transformação ocorre em dois passos:

1. Construímos o **Grafo de Linha** $L(G)$, onde cada vértice representa uma aresta de G e dois vértices são adjacentes se as arestas correspondentes em G compartilham uma ponta. (Emparelhamento clássico em $G \equiv$ Conjunto Independente em $L(G)$).
2. Construímos o **Quadrado do Grafo de Linha** $L(G)^2$. Neste grafo, conectamos dois vértices se a distância entre eles em $L(G)$ for 1 ou 2.

Encontrar um emparelhamento induzido máximo em G equivale exatamente a encontrar um Conjunto Independente Máximo em $L(G)^2$ (CAMERON, 1989). Como o problema do Conjunto Independente é NP-Difícil para grafos gerais (GAREY; JOHNSON, 1990), essa relação estrutural justifica a dificuldade do MIM.

4.3.4 Abordagens de solução e heurísticas

Dada a natureza NP-Difícil do problema e a ineficácia de métodos exatos para grafos densos ou de grande escala, a literatura concentra-se majoritariamente em algoritmos construtivos gulosos e metaheurísticas.

4.3.4.1 Algoritmo exato

Para instâncias pequenas, a redução acima permite resolver o MIM de forma exata. Constrói-se o grafo de conflitos $H = L(G)^2$ e busca-se nele um conjunto independente máximo por *branch-and-bound*: a cada passo, escolhe-se uma aresta candidata e exploram-se dois ramos, incluí-la ou não na solução. Um ramo é podado quando, mesmo incluindo todas as candidatas restantes, não superaria a melhor solução já encontrada. É a abordagem usada neste trabalho como referência exata (Algoritmo 27), viável apenas para algumas dezenas de vértices.

Algorithm 27: Emparelhamento induzido máximo exato, via conjunto independente máximo em $L(G)^2$ (CAMERON, 1989)

Input: graph $G = (V, E)$
Output: maximum induced matching M^*
(1) build $H = L(G)^2$: one vertex per edge of G; $e \sim f$ in H iff e and f share a vertex or are joined by an edge of G
(2) $M^* := \emptyset$
(3) BRANCH$(\emptyset, E)$
(4) **return** M^*

(5) **Procedure** BRANCH(M, C)
(6) | **[comment: M: current solution; C: candidate edges not in conflict with M]**
(7) | **if** $|M| + |C| \leq |M^*|$ **then return [comment: bound]**
(8) | **if** $C = \emptyset$ **then**
(9) | | $M^* := M$
(10) | | **return**
(11) | **end**
(12) | choose $e \in C$
(13) | BRANCH$(M \cup \{e\}, C \setminus (N_H(e) \cup \{e\}))$ **[comment: include e]**
(14) | BRANCH$(M, C \setminus \{e\})$ **[comment: exclude e]**

4.3.4.2 Heurísticas gulosas

Dada a equivalência entre o problema do Emparelhamento Induzido Máximo em G e o Conjunto Independente Máximo no grafo transformado $L(G)^2$, as heurísticas gulosas clássicas da literatura de conjuntos independentes são naturalmente adaptadas para este contexto.

O procedimento geral consiste em selecionar iterativamente uma aresta $e \in E$, adicioná-la à solução e remover sua vizinhança induzida do grafo. As variantes diferem no critério de seleção da aresta:

- **Guloso Aleatório (Random Greedy):** Seleciona a próxima aresta uniformemente ao acaso dentre as disponíveis. Duckworth, Wormald e Zito (2002) analisaram o comportamento assintótico desta abordagem em grafos cúbicos aleatórios, demonstrando que, apesar de sua simplicidade, ela fornece uma linha de base (*baseline*) essencial para a avaliação de métodos mais sofisticados.
- **Guloso de Grau Mínimo (Min-Degree Greedy):** A heurística prioriza a seleção da aresta que possui o **menor grau induzido** (ou seja, a aresta cuja remoção implica a eliminação do menor número de outras arestas candidatas). Esta abordagem é uma adaptação direta da heurística *Min-Degree* (ou *Greedy*) para o problema do Conjunto Independente Máximo. Halldórsson e Radhakrishnan (1994) estabeleceram os fundamentos teóricos para essa estratégia, demonstrando que priorizar vértices de baixo grau fornece garantias de aproximação superiores em grafos esparsos e de grau limitado.

No contexto do MIM, essa lógica traduz-se em escolher a aresta $e = (u, v)$ que minimiza a soma dos graus de seus extremos ou o tamanho de sua vizinhança em $L(G)^2$.

As duas variantes compartilham a mesma estrutura e diferem apenas no critério de seleção, conforme o Algoritmo 28.

Algorithm 28: Heurísticas gulosas para emparelhamento induzido: aleatória e de grau mínimo (DUCKWORTH; WORMALD; ZITO, 2002; HALLDóRSSON; RADHAKRISHNAN, 1994)

Input: graph $G = (V, E)$; selection rule RANDOM or MINDEGREE
Output: maximal induced matching M

(1) build $H = L(G)^2$
(2) $M := \emptyset$; $C := E$
(3) **while** $C \neq \emptyset$ **do**
(4) | **if** *rule is* RANDOM **then**
(5) | | choose $e \in C$ uniformly at random
(6) | **else**
(7) | | choose $e \in C$ with minimum degree in $H[C]$
(8) | **end**
(9) | $M := M \cup \{e\}$
(10) | $C := C \setminus (N_H(e) \cup \{e\})$
(11) **end**
(12) **return** M

4.3.4.3 Metaheurísticas

Para superar os ótimos locais frequentemente encontrados pelos métodos gulosos, abordagens iterativas de busca local têm sido aplicadas.

Fürst, Leichter e Rautenbach (2017) investigaram formalmente o poder da busca local para o Emparelhamento Induzido Máximo. A estratégia analisada consiste em iniciar com um emparelhamento induzido qualquer (ou vazio) e tentar aumentá-lo iterativamente através de melhorias locais simples. Eles focaram na análise de grafos k-regulares e demonstraram que, embora a busca local possa ficar presa em ótimos locais longe da solução global em grafos gerais, ela oferece garantias de desempenho interessantes para classes específicas (como grafos livres de garras ou C_4-free). O algoritmo de busca local proposto tenta adicionar arestas que não violem a restrição induzida e, se necessário, remover um número pequeno de arestas existentes para acomodar a nova inclusão, um processo análogo às trocas (*k-swaps*) em outros problemas combinatórios.

Tabela 6: Algoritmos para problema de emparelhamento induzido máximo em grafos gerais

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Guloso Aleatório	$O(E^2)$*	Heurística	Sim	Seleciona arestas uniformemente ao acaso e remove suas vizinhanças induzidas.
Guloso de grau mínimo	$O(E^2)$*	Heurística	Sim	Seleciona a aresta cuja remoção elimina o menor número de arestas vizinhas.
Exato (conjunto independente)	exponencial	Exato	Sim	Busca por conjunto independente máximo em $L(G)^2$ via *branch-and-bound*; viável apenas em instâncias pequenas.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 9. * Complexidade das implementações avaliadas, que constroem explicitamente o grafo de conflitos $L(G)^2$.

4.4 Outros problemas

No futuro, serão inclusos neste trabalho outros problemas de emparelhamento em grafos gerais, como o de b-emparelhamento, emparelhamento em matroid, e o Stable Roommates Problem.

5 Metodologia

Este capítulo descreve o ambiente de testes construído para implementar e comparar os algoritmos dos capítulos anteriores. São apresentados a arquitetura do ambiente, a interface padrão de cada problema, os geradores de instância, o protocolo de medição, os critérios de comparação, a máquina utilizada e a origem e a adaptação de cada algoritmo, de modo que os resultados do capítulo seguinte sejam reprodutíveis e as comparações, defensáveis.

5.1 Arquitetura do ambiente de testes

O ambiente é separado em três camadas:

- **Repositório de código de terceiros:** Contém os códigos de algoritmos obtidos de repositórios públicos, em sua maioria do GitHub. Estes códigos estão intactos, exatamente da forma que estão publicados. Nenhuma alteração foi feita a eles, nem mesmo com intuito de corrigir possíveis problemas, os quais serão mencionados posteriormente. As bibliotecas maduras usadas como referência (NetworkX, SciPy, igraph e lap) não ficam nesta camada: são instaladas pelo gerenciador de pacotes do Python, com versões fixadas.
- **Adaptadores/*wrappers*:** Como os algoritmos divergem quanto aos formatos de entrada e saída, e até mesmo quanto à linguagem, essa camada foi criada com o intuito de traduzir cada um dos algoritmos para o formato padrão do trabalho, a fim de manter o ambiente homogêneo para fim de testes. Quando o código original não pode ser usado diretamente, a reimplementação também fica nesta camada, de modo que o original permanece intacto no repositório.
- **Módulo de medição:** É a parte que executa e mede os testes, responsável por cronometragem, verificação e gravação dos resultados. Para cada problema, um *script* de *benchmark* gera as instâncias e aciona este módulo, que executa todos os algoritmos por meio dos seus *wrappers*. Isso garante que todos sejam submetidos ao mesmo protocolo, e que possam ser feitas comparações entre algoritmos sob condições idênticas.

5.2 Interface padrão dos problemas

Cada problema possui uma interface padrão à qual todos os seus algoritmos são adaptados (Tabela 7). É ela que torna a comparação justa: todo algoritmo de um mesmo problema recebe exatamente a mesma instância e produz a saída no mesmo formato, de modo que a única variável entre eles é o próprio método.

Tabela 7: Interface padrão de entrada e saída por problema

Problema	**Entrada**	**Saída**
Cardinalidade máxima (bipartido)	vértices de cada lado e lista de arestas	tamanho e emparelhamento
Atribuição linear	matriz de custos $n \times n$	custo total e atribuição
Atribuição gargalo	matriz de custos $n \times n$	custo gargalo e atribuição
Casamento estável	listas de preferência de ambos os lados	emparelhamento
Cardinalidade máxima (geral)	número de vértices e lista de arestas	tamanho e emparelhamento
Peso máximo (geral)	número de vértices e arestas com peso	peso total e emparelhamento
Emparelhamento induzido máximo	número de vértices e lista de arestas	tamanho e emparelhamento

5.3 Geradores de instância

As instâncias são geradas por construtores parametrizados pelo tamanho n e por uma semente fixa, o que as torna determinísticas e reprodutíveis. Para cada problema definiram-se famílias com características distintas, de modo a expor o comportamento dos algoritmos sob diferentes estruturas de entrada: para cardinalidade máxima em grafos bipartidos, grafos esparsos (com e sem emparelhamento perfeito) e densos; para atribuição linear e gargalo, matrizes densas aleatórias, matrizes estruturadas (custo em função da distância entre índices) e matrizes com poucos valores distintos (muitos empates); para casamento estável, preferências aleatórias, correlacionadas (*ranking* de popularidade compartilhado, que gera contenção) e cíclicas; e, para grafos gerais e emparelhamento induzido máximo, grafos aleatórios de densidade controlada, ponderados quando aplicável.

5.4 Protocolo de medição

Um protocolo de medição foi criado a fim de reduzir o quanto fontes de ruído externas afetam os resultados dos testes. A unidade de medição é a **célula**: a combinação de um algoritmo, uma instância e um tamanho n. Cada célula é executada uma vez para aquecimento e, em seguida, cinco vezes com cronometragem. Algumas das decisões que compõem esse protocolo:

- **Execução de aquecimento:** A primeira execução de cada célula é descartada da cronometragem, para que as repetições medidas não paguem custos de importação, alocação inicial e caches frios. O resultado dessa execução é o utilizado na verificação de qualidade.
- **Coletor de lixo:** É desligado durante a execução de uma célula inteira, e religado entre células, a fim de não comprometer o resultado da execução.
- **Mediana:** Cada célula reporta a mediana dos tempos, ao invés da média, para dar mais robustez com relação a interrupções momentâneas.
- **Limite de tempo por célula:** Impede que um algoritmo defeituoso inviabilize toda a suíte de testes. Fez-se necessário por conta de uma situação observada durante os testes, que será discutida na seção de resultados. O limite, de 120 segundos, é aplicado à execução de aquecimento: se ela o excede, é interrompida, a célula é marcada como estourada e os tamanhos maiores daquele algoritmo não são executados, pois também estourariam. A exceção é o problema de atribuição, em que todos os tamanhos são executados mesmo após um estouro, pois um dos algoritmos falha de forma não-monótona no tamanho da entrada. Por decisão deliberada, o limite é verificado apenas em torno do aquecimento, e não a cada operação do algoritmo, para não adicionar custo à região cronometrada.

5.5 Critérios de comparação

Comparar tempo de execução entre implementações em linguagens diferentes é intrinsecamente injusto: parte da diferença reflete a linguagem, não o algoritmo. Para que as comparações sejam válidas, adotaram-se quatro critérios complementares.

1. Implementações em linguagem interpretada (ex: Python) e as de núcleo compilado (ex: C++) são apresentadas em tabelas separadas. Bibliotecas escritas inteiramente em linguagem interpretada, como o NetworkX, ficam na tabela das interpretadas, pois nesse caso a comparação de tempo com elas é legítima. Há ainda casos intermediários: algoritmos escritos como laços em Python sobre vetores da biblioteca NumPy, cujo núcleo é compilado, como os Húngaros `hunalgorithm` e `mayorx` e o Gale-Shapley numérico. Estes são classificados como interpretados, pois o algoritmo em si é executado pelo interpretador.
2. As tabelas mostram complexidade assintótica ao lado do tempo medido, permitindo a comparação do teórico vs real, principalmente entre grafos de diferentes densidades.

3. Para cada algoritmo, é estimado o **expoente empírico do crescimento**. Supondo que o tempo cresça como $t(n) \approx C \cdot n^k$, aplicar o logaritmo resulta em

$$\log t(n) = \log C + k \log n, \tag{18}$$

isto é, uma reta de inclinação k no gráfico do logaritmo do tempo pelo logaritmo do tamanho. O expoente é estimado pela inclinação da reta ajustada por mínimos quadrados aos pontos medidos $(x_i, y_i) = (\log n_i, \log t_i)$:

$$k = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}. \tag{19}$$

Esse dado, ao contrário do tempo absoluto, é comparável entre linguagens: a linguagem altera apenas a constante C, o que desloca a reta verticalmente sem mudar sua inclinação. Junto ao expoente é reportado o coeficiente de determinação R^2, que indica o quanto os pontos de fato seguem uma reta; um R^2 baixo sinaliza que o crescimento não é bem descrito por uma única potência de n na faixa medida.

4. Em problemas com apenas soluções exatas, a qualidade da solução é idêntica entre todos, sendo o método de comparação o tempo de execução. Já nos que têm soluções heurísticas, é apresentado o resultado obtido e a razão entre este e o resultado do algoritmo exato de referência (ou melhor resultado conhecido), permitindo calcular a qualidade da solução.

5.6 Ambiente de execução e reprodutibilidade

Todos os resultados foram obtidos numa única máquina (Tabela 8); medir tudo no mesmo ambiente é o que torna os tempos comparáveis. Três fatores exigiram cuidados, dois da máquina e um da linguagem, todos documentados para permitir reprodução.

1. Por ser um **processador híbrido** (núcleos de desempenho e de eficiência, não equivalentes), a execução foi fixada a um núcleo de desempenho, evitando que a migração entre núcleos alterasse o tempo sem mudança no código.
2. Como a **frequência varia** dinamicamente com a carga, o regulador foi fixado no modo de desempenho, com a máquina conectada à energia e sem aplicações concorrentes.
3. Em Python, o *hash* de cadeias de caracteres é aleatorizado a cada processo, o que altera a ordem de iteração de conjuntos e dicionários entre execuções. Por isso, a semente dessa aleatorização foi fixada (variável `PYTHONHASHSEED`), tornando as execuções reprodutíveis. Isso foi necessário pois foi notada em um dos algoritmos uma grande sensibilidade a essa ordem, o que será detalhado no capítulo de resultados.

Tabela 8: Especificação da máquina de medição

Componente	**Especificação**
Processador	Intel Core 7 150U (10 núcleos físicos, 12 *threads*)
Topologia	híbrida: 2 núcleos de desempenho e 8 de eficiência
Memória	33 GB
Sistema operacional	Ubuntu 24.04.2 LTS (kernel 7.0.0)
Linguagem	Python 3.12.3
Bibliotecas	NumPy 2.5.2, SciPy 1.18.0, NetworkX 3.6.1, igraph 1.0.0, lap 0.5.13

5.7 Origem, adaptação e verificação dos algoritmos

A maior parte dos algoritmos foi obtida de implementações públicas, em vez de reescrita. Isso é coerente com o objetivo de comparar métodos presentes na literatura, reduz o risco de erros próprios e permite

avaliar a qualidade das implementações efetivamente disponíveis. Foram empregadas quatro estratégias de adaptação, conforme natureza do código:

1. Carregamento direto, quando o original é utilizável na linguagem do projeto;
2. Encapsulamento, para métodos disponíveis em bibliotecas maduras;
3. Reimplementação fiel, quando o original não é utilizável diretamente, seja por estar em outra linguagem ou versão, ou por conter efeitos colaterais, seguindo a lógica original com a fonte documentada;
4. Implementação própria a partir da fonte bibliográfica, para as heurísticas e alguns exatos sem implementação pública adequada, algoritmos de descrição curta cuja reimplementação é mais confiável que um repositório de qualidade incerta.

A Tabela 9 registra o tipo e a origem de cada algoritmo, e aponta o pseudocódigo correspondente nos capítulos anteriores.

Tabela 9: Proveniência dos algoritmos avaliados

Algoritmo	**Tipo**	**Pseudocódigo**	**Origem**
Cardinalidade máxima — grafos bipartidos			
Caminhos aumentantes	Exato	Alg. 6	repositório `wbchristerson/perfect-matchings`
Hopcroft-Karp	Exato	Alg. 8	repositório `sofiat-olaosebikan/hopcroftkarp`
Edmonds-Karp (fluxo)	Exato	Alg. 7	repositório `Maxflow-Algorithms` (reimplementado)
Hopcroft-Karp (NetworkX)	Referência	Alg. 8	biblioteca NetworkX (interpretada)
Emparelhamento bipartido (igraph)	Referência	—	biblioteca igraph (núcleo compilado)
Gulosa	Heurística	Alg. 9	implementação própria (CORMEN et al., 2009)
Karp-Sipser	Heurística	Alg. 10	implementação própria (KARP; SIPSER, 1981)
Atribuição linear			
Húngaro (munkres)	Exato	Alg. 11	repositório `bmc/munkres`
Húngaro (benchaplin)	Exato	Alg. 11	repositório `benchaplin/hungarian-algorithm`
Húngaro (hunalgorithm)	Exato	Alg. 11	repositório `pbertoni/HungarianAlgorithm`
Húngaro (mayorx)	Exato	Alg. 11	repositório `mayorx/hungarian-algorithm`
Caminhos mínimos sucessivos	Exato	Alg. 14	reimplementado a partir de repositório
SciPy	Referência	Alg. 13	biblioteca SciPy (núcleo compilado)
Jonker-Volgenant	Referência	Alg. 13	biblioteca lap (núcleo compilado)
Atribuição gargalo			
Threshold	Exato	Alg. 16	reimplementado (original em JavaScript)
Caminhos aumentantes	Exato	Alg. 18	implementação própria (BURKARD; DELL'AMICO; MARTELLO, 2009)
Casamento estável			
Gale-Shapley (orientado a objetos)	Exato	Alg. 20	pacote `gale-shapley-algorithm`
Gale-Shapley (numérico)	Exato	Alg. 20	pacote `gale-shapley-algorithm`
Gale-Shapley (Lattas)	Exato	Alg. 20	reimplementado (repositório `StableMarriages`)
Grafos gerais			
Blossom/Edmonds (cardinalidade)	Exato	Alg. 21	repositório `adharshkamath/Edmonds-Algorithm`
van Rantwijk (peso máximo)	Exato	Alg. 25	`mwmatching.py` de J. van Rantwijk
NetworkX (cardinalidade e peso)	Referência	Alg. 21 e 25	biblioteca NetworkX (interpretada)
Path Growing	Heurística	Alg. 26	implementação própria (DRAKE; HOUGARDY, 2003)
Emparelhamento induzido máximo			
Exato (conjunto independente)	Exato	Alg. 27	implementação própria (CAMERON, 1989)
Guloso aleatório	Heurística	Alg. 28	implementação própria
Guloso de grau mínimo	Heurística	Alg. 28	implementação própria

Os endereços completos constam no cabeçalho de cada adaptador, no código-fonte. Cabe uma ressalva de linhagem: a rotina de emparelhamento de peso máximo em grafos gerais do NetworkX é uma adaptação

do mesmo código de van Rantwijk também avaliado de forma independente; compará-las mede o efeito da engenharia de biblioteca sobre um mesmo algoritmo, e não duas abordagens distintas.

Por fim, nenhuma implementação própria ou reimplementada foi incorporada sem **verificação independente**: os métodos exatos foram confrontados com soluções por força bruta (em instâncias pequenas) e com bibliotecas consolidadas; as heurísticas, quanto à validade da solução, a não superarem o ótimo e a respeitarem suas garantias de aproximação. Além disso, os benchmarks já existentes ao início do trabalho foram migrados para o ambiente unificado sob uma salvaguarda de não-regressão: a qualidade de saída de cada um foi registrada antes da migração e exigiu-se que a nova execução a reproduzisse exatamente, de modo que a unificação não alterasse nenhuma conclusão.

6 Resultados e discussão

Este capítulo apresenta e discute os resultados obtidos com o ambiente de testes descrito no capítulo anterior. Para cada problema, apresentam-se **todas as famílias de instância** definidas na Seção 5.4, de modo a expor como a estrutura da entrada afeta o desempenho e a qualidade. As tabelas de tempo separam as implementações em linguagem interpretada das de núcleo compilado, conforme a Seção 5.5, e seguem sempre a mesma ordem: tempo em Python, tempo compilado (quando houver) e qualidade. Todos os valores são gerados diretamente a partir dos dados medidos. A Tabela 9 relaciona cada algoritmo destas tabelas ao seu pseudocódigo e à sua origem. Ao final, discutem-se os defeitos encontrados em implementações públicas e o crescimento empírico dos tempos frente à complexidade teórica.

6.1 Cardinalidade máxima em grafos bipartidos

As tabelas a seguir apresentam tempo e qualidade para os três grafos: dois esparsos (um sem e um com emparelhamento perfeito) e um denso.

Tabela 10: Tempo de execução em Python — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	$O(V + E)$	3.65	13.93	30.60	54.24	83.43
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.55	1.10	1.64	2.16	2.73
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	$O(V^2)$	9.56	65.27	216.18	519.03	1024.37
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.57	1.20	1.55	2.02	2.80
Gulosa	heuristica	$O(E)$	$O(V)$	0.01	0.01	0.02	0.03	0.03
Karp-Sipser	heuristica	$O(V \cdot E)$	$O(V + E)$	0.16	0.44	0.88	1.50	2.30

Tabela 11: Tempo de execução em compilado (C/C++) — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.09	0.14	0.20	0.26	0.31

Tabela 12: Qualidade da solução — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n

Algoritmo	**Tipo**	**Tempo**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	40	80	120	160	200
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	40	80	120	160	200
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	40	80	120	160	200
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	40	80	120	160	200
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	40	80	120	160	200
Gulosa	heuristica	$O(E)$	37	76	116	156	195
Karp-Sipser	heuristica	$O(V \cdot E)$	40	80	120	160	200

Tabela 13: Tempo de execução em Python — Grafo 2 — Esparso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	$O(V + E)$	6.08	25.24	58.92	75.27	419.70
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.58	1.08	1.67	2.17	3.72
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	$O(V^2)$	9.12	62.41	208.19	498.99	984.10
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.47	0.93	1.36	1.76	2.91
Gulosa	heuristica	$O(E)$	$O(V)$	0.01	0.01	0.02	0.02	0.03
Karp-Sipser	heuristica	$O(V \cdot E)$	$O(V + E)$	0.20	0.59	1.23	2.11	2.53

Tabela 14: Tempo de execução em compilado (C/C++) — Grafo 2 — Esparso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.08	0.13	0.20	0.25	0.31

Tabela 15: Qualidade da solução — Grafo 2 — Esparso | COM perfeito | MCM = n

Algoritmo	**Tipo**	**Tempo**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	50	100	150	200	250
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	50	100	150	200	250
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Gulosa	heuristica	$O(E)$	43	86	130	170	216
Karp-Sipser	heuristica	$O(V \cdot E)$	50	100	150	200	250

Tabela 16: Tempo de execução em Python — Grafo 3 — Denso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	$O(V + E)$	2.08	19.12	34.10	39.79	259.82
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	$O(V + E)$	1.14	2.41	3.99	6.19	7.53
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	$O(V^2)$	12.19	75.08	238.34	556.37	1077.90
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	1.51	3.15	4.84	6.94	8.66
Gulosa	heuristica	$O(E)$	$O(V)$	0.03	0.06	0.08	0.11	0.14
Karp-Sipser	heuristica	$O(V \cdot E)$	$O(V + E)$	0.49	1.35	3.87	4.65	6.86

Tabela 17: Tempo de execução em compilado (C/C++) — Grafo 3 — Denso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.20	0.36	0.61	0.85	1.01

Tabela 18: Qualidade da solução — Grafo 3 — Denso | COM perfeito | MCM = n

Algoritmo	Tipo	Tempo	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	50	100	150	200	250
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	50	100	150	200	250
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Gulosa	heuristica	$O(E)$	47	93	142	191	235
Karp-Sipser	heuristica	$O(V \cdot E)$	50	100	150	200	249

Entre os algoritmos exatos, a hierarquia de desempenho é a mesma nas três instâncias: o Edmonds-Karp, que reduz o emparelhamento a um problema de fluxo, é mais de duas ordens de grandeza mais lento que o Hopcroft-Karp, que agrupa os caminhos aumentantes em fases. A distância é ainda maior no grafo denso, onde o número de arestas amplifica o custo de reconstruir a busca a cada aumento. A implementação de núcleo compilado (igraph) é a mais rápida em todas as instâncias, como esperado.

Quanto às heurísticas, o compromisso entre qualidade e tempo se confirma, mas com uma nuance revelada pela variação entre instâncias: a heurística gulosa, embora seja a mais rápida de todo o conjunto, tem sua pior qualidade não no grafo denso, e sim no grafo esparso com emparelhamento perfeito, em que a ordem de varredura das arestas mais facilmente a prende em um emparelhamento maximal pequeno. A heurística de Karp-Sipser, por sua vez, encontra o ótimo em todas as instâncias esparsas e fica a menos de meio ponto percentual dele no grafo denso, resultado da regra dos vértices de grau um, que domina em grafos esparsos.

6.2 Atribuição linear

Apresentam-se as três matrizes de custo: densa uniforme (aleatória), permutação barata e estruturada.

Tabela 19: Tempo de execução em Python — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	$O(n^2)$	0.39	1.98	4.82	15.43	41.61	97.23	239.96	428.17	679.84
Húngaro (benchaplin)	exato	$O(n^3)$	$O(n^2)$	11.11	92.32	391.64	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr
Húngaro (hunalgorithm)	exato	$O(n^3)$	$O(n^2)$	2.10	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc
Húngaro (mayorx/KM)	exato	$O(n^3)$	$O(n^2)$	0.99	2.25	7.49	12.30	21.11	29.30	58.44	58.55	86.71
SSP (flows)	exato	$O(n^4)^\star$	$O(n^2)$	0.45	2.65	6.92	29.97	98.01	235.13	—	—	—

Tabela 20: Tempo de execução em compilado (C/C++) — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
SciPy (linear_sum_assg)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.08	0.23	0.51	0.84	2.19	3.68	5.48
Jonker-Volgenant (lap)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.05	0.07	0.17	0.36	0.61	1.36	2.31	3.73

Tabela 21: Qualidade da solução — Matriz 1 — Densa uniforme | custo in [1,100]

Algoritmo	Tipo	Tempo	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	102	146	168	217	238	244	258	275	300
Húngaro (benchaplin)	exato	$O(n^3)$	102	146	168	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr
Húngaro (hunalgorithm)	exato	$O(n^3)$	102	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc
Húngaro (mayorx/KM)	exato	$O(n^3)$	102	146	168	217	238	244	258	275	300
SSP (flows)	exato	$O(n^4)^\star$	102	146	168	217	238	244	—	—	—
SciPy (linear_sum_assg)	baseline	$O(n^3)$	102	146	168	217	238	244	258	275	300
Jonker-Volgenant (lap)	baseline	$O(n^3)$	102	146	168	217	238	244	258	275	300

Tabela 22: Tempo de execução em Python — Matriz 2 — Permutação barata | ótimo = n (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	$O(n^2)$	0.11	0.32	0.62	1.61	3.44	5.87	13.12	23.18	35.94
Húngaro (benchaplin)	exato	$O(n^3)$	$O(n^2)$	2.13	7.57	15.76	41.53	91.41	170.38	398.76	728.52	1163.23
Húngaro (hunalgorithm)	exato	$O(n^3)$	$O(n^2)$	0.14	0.37	0.72	1.81	4.35	7.42	16.75	28.39	44.60
Húngaro (mayorx/KM)	exato	$O(n^3)$	$O(n^2)$	0.42	0.80	1.23	2.12	3.39	4.84	8.22	12.34	17.08
SSP (flows)	exato	$O(n^4)^\star$	$O(n^2)$	0.40	2.15	6.23	25.72	86.22	212.71	—	—	—

Tabela 23: Tempo de execução em compilado (C/C++) — Matriz 2 — Permutação barata | ótimo = n (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
SciPy (linear_sum_assg)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.17	0.35	0.61	1.35	2.27	3.54
Jonker-Volgenant (lap)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.16	0.32	0.56	1.18	2.04	3.20

Tabela 24: Qualidade da solução — Matriz 2 — Permutação barata | ótimo = n

Algoritmo	Tipo	Tempo	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (benchaplin)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (hunalgorithm)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (mayorx/KM)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
SSP (flows)	exato	$O(n^4)^\star$	10	20	30	50	75	100	—	—	—
SciPy (linear_sum_assg)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250
Jonker-Volgenant (lap)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250

Tabela 25: Tempo de execução em Python — Matriz 3 — Estruturada | custo = |i-j|+1 (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	$O(n^2)$	0.10	0.32	0.62	1.64	3.38	5.96	13.21	23.29	36.16
Húngaro (benchaplin)	exato	$O(n^3)$	$O(n^2)$	2.15	7.58	15.80	41.51	91.39	170.36	396.75	726.02	1157.10
Húngaro (hunalgorithm)	exato	$O(n^3)$	$O(n^2)$	0.13	0.36	0.71	1.78	4.31	7.31	16.45	27.90	43.83
Húngaro (mayorx/KM)	exato	$O(n^3)$	$O(n^2)$	0.41	0.80	1.21	2.12	3.40	4.83	8.27	12.35	17.01
SSP (flows)	exato	$O(n^4)^\star$	$O(n^2)$	0.39	2.17	6.28	26.00	87.68	214.55	—	—	—

Tabela 26: Tempo de execução em compilado (C/C++) — Matriz 3 — Estruturada | custo = |i-j|+1 (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
SciPy (linear_sum_assg)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.17	0.35	0.58	1.30	2.23	3.50
Jonker-Volgenant (lap)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.16	0.32	0.55	1.18	2.04	3.19

Tabela 27: Qualidade da solução — Matriz 3 — Estruturada | custo = |i-j|+1

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (benchaplin)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (hunalgorithm)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (mayorx/KM)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
SSP (flows)	exato	$O(n^4)^\star$	10	20	30	50	75	100	—	—	—
SciPy (linear_sum_assg)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250
Jonker-Volgenant (lap)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250

Todos os algoritmos que concluem atingem o custo ótimo, de modo que a comparação recai sobre o tempo. Entre as quatro implementações do método Húngaro, obtidas de autores distintos para exatamente o mesmo algoritmo, observa-se variação de mais de uma ordem de grandeza — evidência de que a qualidade da implementação, e não apenas o algoritmo, determina o desempenho na prática. As implementações de núcleo compilado (SciPy e Jonker-Volgenant) são as mais rápidas em todas as matrizes.

O ponto mais instrutivo, porém, só aparece ao comparar as instâncias: os defeitos das implementações `benchaplin` e `hunalgorithm` manifestam-se **exclusivamente na matriz densa aleatória**. Nas matrizes estruturadas, que produzem padrões de custo mais regulares, ambas concluem normalmente. Essa sensibilidade ao conteúdo da entrada — em que a mesma implementação ora funciona, ora falha, conforme a estrutura da matriz — é discutida em conjunto na Seção 6.7.

6.3 Atribuição gargalo

Apresentam-se as três matrizes de custo, para os dois algoritmos exatos.

Tabela 28: Tempo de execução em Python — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	$O(n^2)$	0.10	0.50	1.44	4.48	13.50	32.06	109.54	257.24
Caminhos aumentantes	exato	$O(n^3)$	$O(n^2)$	0.05	0.23	0.62	2.10	5.43	11.32	32.65	68.21

Tabela 29: Qualidade da solução — Matriz 1 — Densa uniforme | custo in [1,100]

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	26	18	18	13	8	7	5	4
Caminhos aumentantes	exato	$O(n^3)$	26	18	18	13	8	7	5	4

Tabela 30: Tempo de execução em Python — Matriz 2 — Estruturada | custo = |i-j|+1 (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	$O(n^2)$	0.10	0.55	1.42	6.51	23.44	51.67	192.45	438.60
Caminhos aumentantes	exato	$O(n^3)$	$O(n^2)$	0.03	0.11	0.23	0.61	1.35	2.46	5.63	10.13

Tabela 31: Qualidade da solução — Matriz 2 — Estruturada | custo = |i-j|+1

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	1	1	1	1	1	1	1	1
Caminhos aumentantes	exato	$O(n^3)$	1	1	1	1	1	1	1	1

Tabela 32: Tempo de execução em Python — Matriz 3 — Poucos valores | muitos empates (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	$O(n^2)$	0.05	0.27	0.80	2.97	9.97	23.25	79.08	188.72
Caminhos aumentantes	exato	$O(n^3)$	$O(n^2)$	0.06	0.42	1.33	5.26	15.55	35.09	111.08	253.58

Tabela 33: Qualidade da solução — Matriz 3 — Poucos valores | muitos empates

Algoritmo	Tipo	Tempo	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	5	1	1	1	1	1	1	1
Caminhos aumentantes	exato	$O(n^3)$	5	1	1	1	1	1	1	1

Ambos os algoritmos produzem sempre o mesmo custo gargalo ótimo, o que também serve como verificação cruzada de corretude. O tempo, no entanto, revela o achado mais interessante deste problema, e que ilustra bem por que vale examinar todas as instâncias: **a ordem entre os dois algoritmos se inverte conforme a matriz**. Nas matrizes densa e estruturada, o método de caminhos aumentantes, construtivo, é muito mais rápido que o de *threshold*. Na matriz com poucos valores distintos, contudo, o *threshold* passa a ser o mais rápido: como sua busca binária opera sobre o conjunto de valores distintos da matriz, poucos valores tornam essa busca curtíssima, enquanto o método construtivo não se beneficia da mesma forma. A conclusão de qual método é preferível, portanto, depende da estrutura esperada da entrada — uma nuance que uma única instância representativa teria ocultado.

6.4 Casamento estável

Apresentam-se os três perfis de preferência: aleatório, correlacionado e cíclico.

Tabela 34: Tempo de execução em Python — Perfil 1 — Aleatório | permutações uniformes (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	$O(n^2)$	7.54	94.14	321.96	692.09	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	$O(n^2)$	0.44	1.57	3.36	5.52	21.70	50.59	95.89
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	$O(n^2)$	0.31	1.29	2.85	5.08	20.07	52.20	100.25

Tabela 35: Qualidade da solução — Perfil 1 — Aleatório | permutações uniformes

Algoritmo	Tipo	Tempo	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	0 =ref	0 =ref	0 =ref	0 =ref	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref

Tabela 36: Tempo de execução em Python — Perfil 2 — Correlacionado | ranking de popularidade (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	$O(n^2)$	43.60	589.68	2826.56	8665.93	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	$O(n^2)$	0.75	2.76	6.22	10.87	43.05	101.47	188.56
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	$O(n^2)$	0.66	3.48	10.24	21.99	144.99	482.24	1160.66

Tabela 37: Qualidade da solução — Perfil 2 — Correlacionado | ranking de popularidade

Algoritmo	Tipo	Tempo	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	0 =ref	0 =ref	0 =ref	0 =ref	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref

Tabela 38: Tempo de execução em Python — Perfil 3 — Cíclico | deslocamento estruturado (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	$O(n^2)$	2.44	14.89	46.06	104.25	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	$O(n^2)$	0.39	1.35	3.00	5.16	20.25	47.17	88.20
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	$O(n^2)$	0.21	0.74	1.60	2.80	10.95	27.22	51.22

Tabela 39: Qualidade da solução — Perfil 3 — Cíclico | deslocamento estruturado

Algoritmo	Tipo	Tempo	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	0 =ref	0 =ref	0 =ref	0 =ref	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref

Pela unicidade do emparelhamento estável ótimo para os pretendentes, as três variantes produzem exatamente o mesmo resultado em todos os perfis — o que as tabelas de qualidade confirmam, com estabilidade e concordância em todas as células. A qualidade, portanto, não as distingue, e a comparação recai sobre o tempo. A variante orientada a objetos, embora teoricamente quadrática, é substancialmente mais lenta e inviável nos tamanhos maiores, em todos os perfis, por conta do sobrecusto de reconstruir estruturas a cada rodada — comportamento quantificado na Seção 6.8. As variantes que operam sobre vetores e dicionários escalam sem dificuldade até os maiores tamanhos avaliados.

6.5 Emparelhamento em grafos gerais

Cobrem-se os dois problemas de grafos gerais — cardinalidade máxima e peso máximo —, cada um em uma instância esparsa e uma densa.

Tabela 40: Tempo de execução em Python — Cardinalidade — esparso (grau ˜3) (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	$O(V^2)$	1.66	3.21	3.97	20.36	43.90	103.46
NetworkX (cardinalidade)	baseline	$O(V^3)$	$O(V + E)$	0.40	0.63	0.74	1.41	2.28	3.15

Tabela 41: Qualidade da solução — Cardinalidade — esparso (grau ˜3)

Algoritmo	Tipo	Tempo	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	5	9	13	23	31	43
NetworkX (cardinalidade)	baseline	$O(V^3)$	5	9	13	23	31	43

Tabela 42: Tempo de execução em Python — Cardinalidade — denso (p=0.3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	$O(V^2)$	1.60	13.79	35.81	342.60	2862.65	9526.65
NetworkX (cardinalidade)	baseline	$O(V^3)$	$O(V + E)$	0.38	0.47	1.07	1.94	12.25	7.46

Tabela 43: Qualidade da solução — Cardinalidade — denso (p=0.3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	5	10	15	25	37	50
NetworkX (cardinalidade)	baseline	$O(V^3)$	5	10	15	25	37	50

Tabela 44: Tempo de execução em Python — Peso — esparso (grau ˜3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	$O(V + E)$	0.26	0.54	0.86	2.19	3.49	6.30
NetworkX (peso máximo)	baseline	$O(V^3)$	$O(V + E)$	0.59	1.25	1.98	4.87	7.52	13.58
Path Growing	heuristica	$O(E)$	$O(V + E)$	0.03	0.05	0.07	0.11	0.17	0.23

Tabela 45: Qualidade da solução — Peso — esparso (grau ˜3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	309	604	790	1284	1907	2929
NetworkX (peso máximo)	baseline	$O(V^3)$	309	604	790	1284	1907	2929
Path Growing	heuristica	$O(E)$	278	510	732	1030	1492	2407

Tabela 46: Tempo de execução em Python — Peso — denso (p=0.3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	$O(V + E)$	0.25	0.87	2.20	4.83	16.68	26.68
NetworkX (peso máximo)	baseline	$O(V^3)$	$O(V + E)$	0.59	2.38	5.83	13.56	41.85	78.35
Path Growing	heuristica	$O(E)$	$O(V + E)$	0.03	0.08	0.15	0.36	0.78	1.35

Tabela 47: Qualidade da solução — Peso — denso (p=0.3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	309	765	1279	2247	3429	4710
NetworkX (peso máximo)	baseline	$O(V^3)$	309	765	1279	2247	3429	4710
Path Growing	heuristica	$O(E)$	278	642	1215	1934	3113	4329

No caso ponderado, um resultado é digno de nota e mantém-se nas duas densidades: a implementação independente de van Rantwijk é mais rápida que a rotina correspondente do NetworkX, ainda que esta seja uma adaptação daquele mesmo código. A comparação, portanto, não opõe duas abordagens, e sim uma implementação a um descendente empacotado dela; a diferença mede o efeito das camadas de generalidade que a biblioteca acrescenta. A heurística Path Growing é ordens de grandeza mais rápida que os exatos e entrega uma fração elevada do peso ótimo em ambas as densidades, sempre bem acima da sua garantia teórica de metade; a qualidade que alcança é, aliás, um pouco superior nos grafos densos.

6.6 Emparelhamento induzido máximo

Sendo NP-difícil, o emparelhamento induzido máximo tem perfil distinto: o algoritmo exato só é viável em instâncias pequenas, e a qualidade das heurísticas é o eixo central. Apresentam-se as três densidades avaliadas. A referência de qualidade depende do tamanho: até $n = 40$, a razão é calculada contra o algoritmo exato; a partir de $n = 50$, em que o exato é inviável, contra o melhor resultado obtido entre as heurísticas. Nesse segundo regime, uma razão igual a 1 indica apenas a melhor heurística, e não necessariamente o ótimo.

Tabela 48: Tempo de execução em Python — Grafo esparso (grau ˜3) (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	$O(E^2)$	0.16	0.29	1.49	1.99	5.10	74.61	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	$O(V+E)$	0.05	0.05	0.08	0.13	0.18	0.29	0.47	1.50	8.28	27.29
Guloso grau mínimo	heuristica	$O(E^2)$	$O(E^2)$	0.18	0.36	0.78	1.47	1.83	2.78	4.27	18.48	85.90	378.36

Tabela 49: Qualidade da solução — Grafo esparso (grau ˜3)

Algoritmo	Tipo	Tempo	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	2	3	5	5	8	10	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	2	2	3	3	5	7	10	18	41	75
Guloso grau mínimo	heuristica	$O(E^2)$	2	3	5	5	8	10	11	25	49	93

Tabela 50: Tempo de execução em Python — Grafo médio (grau ˜6) (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	$O(E^2)$	0.36	1.07	2.50	4.55	10.56	133.00	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	$O(V+E)$	0.06	0.11	0.17	0.25	0.28	0.40	0.60	2.43	9.02	32.93
Guloso grau mínimo	heuristica	$O(E^2)$	$O(E^2)$	0.47	1.55	3.48	5.50	5.88	11.08	21.08	72.77	327.12	1573.84

Tabela 51: Qualidade da solução — Grafo médio (grau ˜6)

Algoritmo	Tipo	Tempo	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	2	3	4	4	6	8	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	1	2	3	3	5	6	7	15	30	55
Guloso grau mínimo	heuristica	$O(E^2)$	2	3	3	4	5	7	8	19	39	72

Tabela 52: Tempo de execução em Python — Grafo denso (grau ˜12) (mediana, ms)

Algoritmo	Tipo	Tempo	Espaço	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	$O(E^2)$	0.64	2.82	5.62	9.65	17.19	58.30	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	$O(V+E)$	0.07	0.14	0.20	0.29	0.38	1.06	0.77	2.82	11.87	39.37
Guloso grau mínimo	heuristica	$O(E^2)$	$O(E^2)$	0.85	4.00	8.80	14.85	23.40	48.81	74.74	326.03	1314.14	6086.44

Tabela 53: Qualidade da solução — Grafo denso (grau ˜12)

Algoritmo	Tipo	Tempo	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	1	2	3	3	4	5	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	1	1	1	2	3	4	4	9	20	35
Guloso grau mínimo	heuristica	$O(E^2)$	1	2	3	3	4	5	6	14	25	55

O algoritmo exato, que reduz o problema a conjunto independente máximo no grafo de conflitos, resolve as instâncias pequenas em poucos milissegundos — inclusive nas densas, em que o emparelhamento induzido máximo é pequeno e a poda é eficaz —, mas seu custo cresce rapidamente e o torna inviável além de algumas dezenas de vértices, o que justifica empiricamente a necessidade das heurísticas. A comparação entre densidades revela um comportamento não-monótono digno de nota: a heurística de grau mínimo alcança o ótimo tanto nos grafos esparsos quanto nos densos, mas tem sua pior qualidade na densidade *intermediária*. A explicação é que, em grafos muito densos, quase toda aresta conflita com as demais, de modo que o emparelhamento induzido é pequeno e fácil de acertar; é na densidade média que há mais espaço para escolhas localmente boas mas globalmente subótimas. A escolha aleatória, por sua vez, degrada-se de forma mais acentuada com o aumento da densidade. Em todos os casos, a heurística de grau mínimo é consistentemente superior à aleatória, ao custo de um tempo maior, por manter e atualizar o grafo de conflitos.

6.7 Defeitos em implementações públicas

Um resultado que só emerge da execução repetida dos algoritmos — e que um levantamento puramente bibliográfico não revelaria — diz respeito à confiabilidade de implementações públicas de algoritmos clássicos. Duas das quatro implementações do método Húngaro avaliadas apresentam defeitos, ambos manifestados apenas nas matrizes densas aleatórias, como visto nas tabelas de qualidade da atribuição.

A primeira (`hunalgorithm`) falha de forma determinística a partir de um certo tamanho, por exceder um limite interno de iterações previsto em seu próprio código.

A segunda (`benchaplin`) é mais notável, por ser **não-determinística entre execuções**. Sobre exatamente a mesma matriz de entrada, a implementação ora retorna o custo ótimo correto, ora falha com um erro de tipo, ora ingressa em um caminho de execução tão lento que não conclui em tempo praticável. A causa é a dependência do defeito quanto à ordem de iteração de uma coleção de objetos, que a linguagem aleatoriza a cada processo. São três desfechos distintos para a mesma entrada, todos observados. Além de ilustrar de forma vívida o valor de *implementar e medir* sobre apenas catalogar, esse comportamento tem uma consequência metodológica: a fixação da semente de aleatorização, descrita no capítulo anterior, foi necessária justamente para tornar os resultados reprodutíveis.

Optou-se por **reportar** esses defeitos, e não corrigi-los: como o objeto de estudo é o conjunto de implementações efetivamente disponíveis, corrigi-los apagaria um resultado legítimo sobre a qualidade do que se encontra publicado.

6.8 Crescimento empírico frente à complexidade teórica

Como o tempo absoluto não é comparável entre linguagens, a análise de crescimento apoia-se no expoente empírico definido na Seção 5.5: a inclinação da reta ajustada ao gráfico do tempo pelo tamanho, em escala logarítmica. As Figuras 1 e 2 apresentam esses gráficos para o grafo e a matriz densos, e as Tabelas 54 a 58 confrontam o expoente medido com a complexidade teórica.

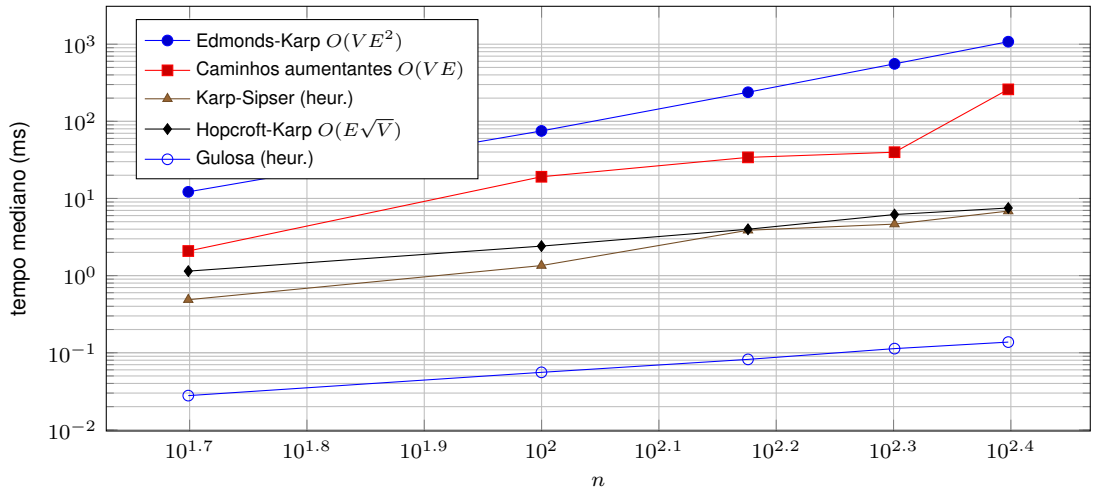


Figura 1: Tempo de execução em escala log-log para o grafo denso (cardinalidade máxima bipartida). A inclinação de cada reta corresponde ao expoente empírico de crescimento; a de Edmonds-Karp, próxima de 3, destaca-se das demais.

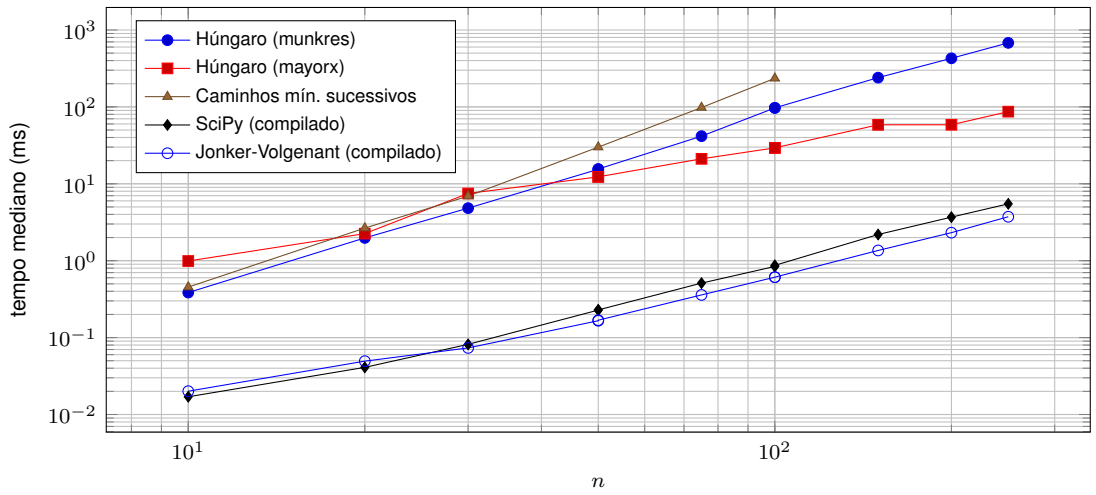


Figura 2: Tempo de execução em escala log-log para a matriz densa (atribuição linear). As duas implementações de núcleo compilado situam-se bem abaixo das interpretadas, mas com inclinação semelhante.

Os expoentes medidos são, em geral, coerentes com a teoria. O caso mais nítido é o do Edmonds-Karp, cujo expoente próximo de três acompanha o comportamento cúbico previsto e o separa visivelmente dos demais na Figura 1. Nas implementações de núcleo compilado, a reta situa-se muito abaixo das interpretadas, mas com inclinação semelhante — exatamente o efeito esperado: a linguagem altera a constante (a posição da reta), não o expoente (a inclinação).

Tabela 54: Expoente empírico de crescimento — Grafo 3 — Denso — COM perfeito — MCM = n

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Aug. Paths (wbchristerson)	$O(V \cdot E)$	2.61	0.917
Hopcroft-Karp (sofiat)	$O(E \cdot \sqrt{V})$	1.20	0.996
Edmonds-Karp (Maxflow-Algs)	$O(V \cdot E^2)$	2.79	0.999
Hopcroft-Karp (NetworkX)	$O(E \cdot \sqrt{V})$	1.09	0.999
Bipartite Match. (igraph)	$O(E \cdot \sqrt{V})$	1.05	0.994
Gulosa	$O(E)$	1.00	1.000
Karp-Sipser	$O(V \cdot E)$	1.68	0.983

Tabela 55: Expoente empírico de crescimento — Matriz 1 — Densa uniforme — custo in [1,100]

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Húngaro (munkres)	$O(n^3)$	2.36	0.999
Húngaro (benchaplin)	$O(n^3)$	3.22	0.998
Húngaro (mayorx/KM)	$O(n^3)$	1.39	0.980
SSP (flows)	$O(n^4)*$	2.71	0.998
SciPy (linear_sum_assg)	$O(n^3)$	1.86	0.994
Jonker-Volgenant (lap)	$O(n^3)$	1.65	0.987

Observa-se ainda que, nos tamanhos avaliados, o expoente medido frequentemente fica *abaixo* do previsto pela complexidade de pior caso, que é conservadora: o pior caso raramente se materializa nas instâncias típicas. A exceção reveladora é a variante orientada a objetos do Gale-Shapley (Tabela 56), cujo expoente medido supera o teórico: o sobrecusto de implementação a torna, na prática, super-quadrática, confirmando por medição a ressalva já registrada sobre ela.

Tabela 56: Expoente empírico de crescimento — Perfil 1 — Aleatório — permutações uniformes

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Gale-Shapley (OOP/pip)	$O(n^2)$	3.28	0.996
Gale-Shapley (numérico/pip)	$O(n^2)$	1.94	0.999
Gale-Shapley (Lattas/dict)	$O(n^3)*$	2.07	0.999

Tabela 57: Expoente empírico de crescimento — Matriz 1 — Densa uniforme — custo in [1,100]

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Threshold (busca binária)	$O(n^3 logn)$	2.61	0.996
Caminhos aumentantes	$O(n^3)$	2.41	0.999

Tabela 58: Expoente empírico de crescimento — Peso — esparso (grau 3)

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
van Rantwijk (mwmatching)	$O(V^3)$	1.40	0.989
NetworkX (peso máximo)	$O(V^3)$	1.36	0.990
Path Growing	$O(E)$	0.88	0.998

7 Trabalhos futuros

Este trabalho construiu um ambiente de testes unificado e o aplicou a sete problemas de emparelhamento em grafos. Algumas extensões ficaram fora do escopo, por limitação de tempo ou por decisão metodológica, e constituem os próximos passos naturais:

- **Algoritmos descritos e não implementados:** Parte dos métodos apresentados nos capítulos de problemas não foi incorporada ao ambiente de testes: o algoritmo de Dinic, para cardinalidade máxima em grafos bipartidos; a variante de caminhos mínimos sucessivos com Dijkstra e potenciais, para a atribuição linear; o método dual, para a atribuição gargalo; o algoritmo de Micali e Vazirani, para cardinalidade máxima em grafos gerais; e a versão otimizada do algoritmo de Edmonds ponderado (GALIL; MICALI; GABOW, 1986). Como o ambiente já define a interface padrão e o protocolo de cada problema, incluí-los exige apenas um novo adaptador por algoritmo.
- **Heurísticas para os demais problemas:** A comparação entre métodos exatos e heurísticos foi feita para cardinalidade máxima em grafos bipartidos, peso máximo em grafos gerais e emparelhamento induzido máximo. Ficaram de fora as heurísticas gulosas descritas para a atribuição linear e a atribuição gargalo, a gulosa ordenada para peso máximo em grafos gerais, as heurísticas gulosa, de Karp-Sipser e de crescimento de caminho para cardinalidade máxima em grafos gerais, e as metaheurísticas de busca local para o emparelhamento induzido (FüRST; LEICHTER; RAUTENBACH, 2017).
- **Contagem de operações elementares:** Avaliou-se instrumentar os algoritmos com contadores de operações, como caminhos aumentantes percorridos e arestas examinadas, uma métrica independente de linguagem e de máquina. A ideia foi descartada porque exigiria manter uma cópia modificada de cada repositório ao lado do original intacto. Uma alternativa a investigar é a instrumentação externa, por ferramentas de rastreamento da própria linguagem, sem alterar o código de terceiros.
- **Separação do efeito da linguagem:** As implementações interpretadas e compiladas foram mantidas em tabelas separadas, e o expoente empírico permitiu compará-las apenas quanto ao crescimento. Implementar um mesmo algoritmo nas duas linguagens permitiria medir diretamente quanto da diferença de tempo se deve à linguagem, e não ao algoritmo.
- **Instâncias reais:** As instâncias utilizadas são sintéticas, geradas por famílias parametrizadas. Aplicar o ambiente a instâncias provenientes das aplicações citadas na introdução verificaria se as conclusões se mantêm fora dos cenários controlados.
- **Comunicação dos defeitos encontrados:** Os defeitos identificados em implementações públicas do método Húngaro (Seção 6.7) podem ser reportados aos seus mantenedores, junto com as instâncias que os reproduzem.
- **Problemas fora do escopo:** As variações NP-difíceis citadas na introdução, como o emparelhamento tridimensional e o problema da atribuição quadrática, e os emparelhamentos em multigrafos e hipergrafos, excluídos na delimitação do estudo.

Referências

AHO, A. V.; HOPCROFT, J. E.; ULLMAN, J. D. *The Design And Analysis of Computer Algorithms*. 1st. ed. Reading, MA: Addison-Wesley, 1974. ISBN 9780201000290.

AHUJA, R. K.; MAGNANTI, T. L.; ORLIN, J. B. *Network flows: theory, algorithms, and applications*. USA: Prentice-Hall, Inc., 1993. ISBN 013617549X.

BURKARD, R.; DELL'AMICO, M.; MARTELLO, S. *Assignment Problems*. USA: Society for Industrial and Applied Mathematics, 2009. ISBN 0898716632.

CAETANO, T. S. et al. Learning graph matching. *IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE*, IEEE Computer Society, United States, v. 31, n. X, December 2009. ISSN 0162-8828.

CAMERON, K. Induced matchings. *Discrete Applied Mathematics*, v. 24, n. 1, p. 97–102, 1989. ISSN 0166-218X. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/0166218X9290275F⟩.

CORMEN, T. H. et al. *Introduction to Algorithms*. 3rd. ed. Cambdrige, MA: The MIT Press, 2009. ISBN 978-0-262-53305-8.

DASGUPTA, C. H. P. S.; VAZIRANI, U. V. *Algorithms*. 1st. ed. New York, NY: McGraw-Hill Education, 2006. ISBN 978-0073523408.

DRAKE, D. E.; HOUGARDY, S. A simple approximation algorithm for the weighted matching problem. *Information Processing Letters*, v. 85, n. 4, p. 211–213, 2003. ISSN 0020-0190. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S0020019002003939⟩.

DUCKWORTH, W.; WORMALD, N.; ZITO, M. Maximum induced matchings of random cubic graphs. *Journal of Computational and Applied Mathematics*, v. 142, n. 1, p. 39–50, 2002. ISSN 0377-0427. Probabilistic Methods in Combinatorics and Combinatorial Optimization. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S0377042701004575⟩.

EDMONDS, J. Paths, trees, and flowers. *Canadian Journal of Mathematics*, v. 17, p. 449–467, 1965.

FAN, W. Graph pattern matching revised for social network analysis. In: *Proceedings of the 15th International Conference on Database Theory*. New York, NY, USA: Association for Computing Machinery, 2012. (ICDT '12), p. 8–21. ISBN 9781450307918. Disponível em: ⟨https://doi.org/10.1145/2274576.2274578⟩.

FüRST, M.; LEICHTER, M.; RAUTENBACH, D. *Locally Searching for Large Induced Matchings*. 2017. Disponível em: ⟨https://arxiv.org/abs/1708.02028⟩.

GABOW, H. N. Data structures for weighted matching and nearest common ancestors with linking. In: *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms*. USA: Society for Industrial and Applied Mathematics, 1990. (SODA '90), p. 434–443. ISBN 0898712513.

GALIL, Z.; MICALI, S.; GABOW, H. An $o(ev \log v)$ algorithm for finding a maximal weighted matching in general graphs. *SIAM J. Comput.*, v. 15, p. 120–130, 02 1986.

GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990. ISBN 0716710455.

HALIM, S.; HALIM, F. *Competitive Programming 3: The New Lower Bound of Programming Contests*. 3rd. ed. [S.l.]: Lulu.com, 2013. Self-published by the authors. ISBN 978-1482852483.

HALLDóRSSON, M.; RADHAKRISHNAN, J. Greed is good: approximating independent sets in sparse and bounded-degree graphs. In: *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*. New York, NY, USA: Association for Computing Machinery, 1994. (STOC '94), p. 439–448. ISBN 0897916638. Disponível em: ⟨https://doi.org/10.1145/195058.195221⟩.

HALLER, S. et al. *A Comparative Study of Graph Matching Algorithms in Computer Vision*. 2022. Disponível em: ⟨https://arxiv.org/abs/2207.00291⟩.

JONKER, R.; VOLGENANT, A. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, v. 38, n. 4, p. 325–340, 1987. Disponível em: ⟨https://doi.org/10.1007/BF02278710⟩.

JUNGNICKEL, D. *Graphs, Networks and Algorithms*. 3rd. ed. Berlin, Germany: Springer, 2008. ISBN 978-354727798.

KARP, R. M.; SIPSER, M. Maximum matching in sparse random graphs. In: *Proceedings of the 22nd Annual Symposium on Foundations of Computer Science*. USA: IEEE Computer Society, 1981. (SFCS '81), p. 364–375. Disponível em: ⟨https://doi.org/10.1109/SFCS.1981.21⟩.

KLEINBERG, J.; TARDOS Éva. *Algorithm Design*. 1st. ed. Boston, MA: Pearson, 2005. ISBN 978-0321295354.

LAWLER, E. L. *Combinatorial Optimization: Networks and Matroids*. New York, NY: Dover Publications, 1976. Reprint edition. ISBN 978-0486414539.

Basic terminology. In: LOVáSZ, L.; PLUMMER, M. (Ed.). *Matching Theory*. North-Holland, 1986, (North-Holland Mathematics Studies, v. 121). p. xxix–xxxiii. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S0304020808736363⟩.

MANBER, U. *Introduction to Algorithms: A Creative Approach*. 1st. ed. Reading, MA: Addison-Wesley, 1989. ISBN 978-0201120370.

MANDULAK, M. et al. Efficient weighted graph matching on gpus. *SC24*, IEEE Computer Society, United States, November 2024. ISSN 0162-8828.

MARTELLO, S.; TOTH, P. Linear assignment problems. In: MARTELLO, S. et al. (Ed.). *Surveys in Combinatorial Optimization*. North-Holland, 1987, (North-Holland Mathematics Studies, v. 132). p. 259–282. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S0304020808732389⟩.

MICALI, S.; VAZIRANI, V. An O(sqrt(—v—) —E—) Algorithm for Finding Maximum Matching in General Graphs. In: . [S.l.: s.n.], 1980. p. 17–27.

ROSEN, K. H.; KREHER, D. L.; STINSON, D. R. *Discrete Mathematics and Its Applications*. 1st. ed. Boca Raton, FL: CRC Press, 1998. ISBN 978-0849339882.

SCHRIJVER, A. *Combinatorial Optimization: Polyhedra and Efficiency*. 1st. ed. Berlin, Germany: Springer, 2004. ISBN 3540204563.

STOCKMEYER, L. J.; VAZIRANI, V. V. Np-completeness of some generalizations of the maximum matching problem. *Information Processing Letters*, v. 15, n. 1, p. 14–19, 1982. ISSN 0020-0190. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/0020019082900771⟩.

SUSSMAN, D. L.; LI, L. *Overview of Graph Matching Challenges and Approaches*. 2021. Acessado em: 26 nov. 2024. Disponível em: ⟨https://www.ll.mit.edu/⟩.

TAYLOR, W. R. Protein structure comparison using bipartite graph matching and its application to protein structure classification. *Molecular and Cellular Proteomics*, American Society for Biochemistry and Molecular Biology, United States, v. 1, n. 4, April 2002. ISSN 1535-9476.

VAZIRANI, V. V. *Approximation algorithms*. Berlin, Heidelberg: Springer-Verlag, 2001. ISBN 3540653678.

ZASLAVSKIY, M.; BACH, F.; VERT, J.-P. A path following algorithm for the graph matching problem. *IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE*, IEEE Computer Society, United States, v. 31, n. 12, December 2009. ISSN 0162-8828.